



REPUBLIC OF RWANDA

KIGALI INDEPENDENT UNIVERSITY ULK

P.O. Box 2280 KIGALI

Cell phone: (+250) 0788304084; 0788303670; 0788304086; 0788304081; 0788302644; 0788302348

E-mail: info@ulk.ac.rw

Website: www.ulk.ac.rw

School name: Science and Technology

Department name: Computer Science

Module code: MSE 9214

Number of credits: 15

Blockchain Technology and Decentralized Applications

Lecturer: Dr. Egide Nkurunziza

Email: nkurunzizaegide@gmail.com

Table of Contents

Chapter 1: Introduction to Blockchain Technology	8
Learning Objectives.....	8
1.0 Setting the Scene: What Is a Blockchain?	8
1.1 History and Evolution.....	9
1.2 Key Terminology.....	11
1.3 Principles of Decentralized Systems: Four Pillars	12
1.4 A Common Point of Confusion: Centralized vs. Distributed vs. Decentralized.....	13
Chapter 1 Recap: Key Takeaways.....	15
Discussion Questions and Model Answers	15
Chapter 2: Cryptographic Foundations	17
Learning Objectives.....	17
2.0 Setting the Scene: Why Does Blockchain Need Cryptography?	17
2.1 Cryptographic Hash Functions	18
2.2 Public-Key Cryptography.....	20
2.3 Digital Signatures	21
2.4 Putting It Together.....	21
Worked Example: Carla Sends a Transaction	22
Chapter 2 Recap: Key Takeaways.....	22
Discussion Questions and Model Answers	23
Chapter 3: How Blockchain Works: A Transaction from Creation to Verification.....	25
Learning Objectives.....	25
3.0 Setting the Scene: Why Does a Transaction Need Verification?	25

3.1 The Transaction Lifecycle, Part 1: From Creation to Network Verification	26
3.2 The Transaction Lifecycle, Part 2: From Block Creation to Permanent Record	26
3.3 The Distributed Ledger: Blocks Linked by Hashes	26
3.4 Key Benefits: What This Process Guarantees	27
Worked Example: Alice Sends Bob 0.5 BTC	27
Chapter 3 Recap: Key Takeaways.....	27
Discussion Questions and Model Answers	28
Chapter 4: Consensus Mechanisms and Mining	29
Learning Objectives.....	29
4.0 The Problem Consensus Mechanisms Solve: Agreement Without a Referee	29
Part 1: Mining and Proof of Work Bitcoin’s Original Security Model.....	30
4.1 Who Decides Which Node Proposes the Next Block?.....	30
4.2 What Is Mining? Competing to Propose the Next Block.....	31
4.3 The Full Mining Process, Step by Step	31
4.4 Rewards and Incentives	33
4.5 Security Strength, Energy Cost	33
4.6 The 51% Attack	34
Worked Example: A Simplified Proof of Work Puzzle	35
Part 2: Beyond Proof of Work Replacing Computation with Economic Commitment ..	35
4.7 The Energy Question: Proof of Work vs. Proof of Stake.....	35
4.8 How Proof of Stake Works.....	35
4.9 Slashing	36

4.10 Ethereum’s “The Merge” (2022).....	36
Part 3: Other Consensus Models Different Trade-offs for Different Needs	36
4.11 Delegated Proof of Stake (DPoS).....	36
4.12 Practical Byzantine Fault Tolerance (PBFT)	37
4.13 Proof of Authority (PoA).....	37
4.14 The Pattern: Speed vs. Centralization	38
Part 4: Comparing the Models.....	38
4.15 Side-by-Side Comparison.....	38
4.16 The Blockchain Trilemma	38
Worked Example: A Miner Finds Block 105.....	39
Chapter 4 Recap: Key Takeaways.....	39
Discussion Questions and Model Answers	40
Chapter 5: Types of Blockchain Networks	42
Learning Objectives.....	42
Blockchain Platforms vs. Blockchain Network Types.....	42
Type 1 Public Blockchain: Open to Anyone, Trusted by Design	43
Type 2 Private Blockchain: One Organization, Full Control	44
Type 3 Consortium (Federated) Blockchain: Shared Control Among Known Partners	45
Type 4 Hybrid Blockchain: The Best of Both Worlds, Selectively	45
Comparing the Four Network Types	46
Advantages and Disadvantages	47
Real-World Applications.....	48

Choosing the Appropriate Network: A Practical Decision Framework.....	49
Worked Example: Picking a Network for a Real Scenario	49
Chapter 5 Recap: Key Takeaways.....	49
Discussion Questions and Model Answers	50
Chapter 6: Blockchain Platforms and Ecosystems	51
Learning Objectives.....	51
Bitcoin: The Pioneer Network, Digital Cash, Deliberately Simple	51
Ethereum: The Programmable Blockchain, “A Computer the Whole World Shares” .	52
Hyperledger and Enterprise Blockchain: When Permissionless Isn’t the Goal	53
Smart Contracts: Concept and Architecture	54
Comparing the Platforms.....	55
Putting It into Practice: Which Platform, and Why?	55
Chapter 6 Recap: Key Takeaways.....	55
Discussion Questions and Model Answers	56
Chapter 7: Development of Decentralized Applications (dApps).....	58
Learning Objectives.....	58
What Makes an App “Decentralized”? The Three-Layer dApp Architecture	58
Introduction to Solidity: Your First Smart Contract.....	59
Development Tools.....	60
Connecting Frontends with Web3.js: Bridging the Browser and the Blockchain	60
The dApp Development Lifecycle: From Idea to Mainnet	61
Worked Example: A Simple Decentralized Voting dApp	61

Chapter 7 Recap: Key Takeaways.....	62
Discussion Questions and Model Answers	62
Chapter 8: Blockchain Security and Privacy.....	64
Learning Objectives.....	64
Immutability and Data Integrity, Revisited: The 2016 DAO Hack	64
Common Smart Contract Vulnerabilities	65
Security Audits and Best Practices: Catching Bugs Before They're Permanent	66
Privacy-Preserving Techniques	67
Scalability and Security: A Persistent Tension	68
Worked Example: Spotting a Reentrancy Vulnerability	68
Chapter 8 Recap: Key Takeaways.....	68
Discussion Questions and Model Answers	69
Chapter 9: Tokenization and Initial Coin Offerings (ICOs).....	71
Learning Objectives.....	71
What Is Tokenization? Turning Ownership into Code	71
Token Standards: ERC-20, ERC-721, ERC-1155.....	72
Initial Coin Offerings (ICOs): How an ICO Works	72
Decentralized Autonomous Organizations (DAOs): Governance Encoded in Smart Contracts.....	73
Non-Fungible Tokens (NFTs) and Their Applications: Beyond Digital Art	73
Worked Example: Designing a Simple Tokenomics Model	74
Chapter 9 Recap: Key Takeaways.....	75
Discussion Questions and Model Answers	75

Chapter 10: Use Cases of Blockchain in Various Industries.....	77
Learning Objectives.....	77
8.1 Finance and Decentralized Finance (DeFi)	77
8.2 Healthcare	78
8.3 Supply Chain Management	79
8.4 Governance and Public Administration.....	80
8.5 Cross-Industry Lessons: When Does Blockchain Actually Help?	81
Chapter 10 Recap: Key Takeaways.....	81
Discussion Questions and Model Answers	82
Appendix A Glossary of Key Terms	83
Appendix B Quick-Reference Comparison Tables	87
Appendix C Additional Self-Check Questions.....	89

Chapter 1: Introduction to Blockchain Technology

History · Key Terminology · Principles of Decentralization

Learning Objectives

By the end of this chapter, you should be able to:

- Define a blockchain in your own words and explain what makes it different from an ordinary shared database.
- Trace the intellectual history of blockchain from 1991 timestamping research through Bitcoin, Ethereum, and today's multi-purpose networks.
- Use the core vocabulary of the field—block, chain, node, ledger, hash function, consensus mechanism, wallet—precisely and confidently.
- Explain the four pillars of decentralized systems: distributed control, transparency, immutability, and trustlessness.
- Distinguish between centralized, distributed, and decentralized systems, and explain why these three words describe different things.

1.0 Setting the Scene: What Is a Blockchain?

A blockchain is a shared, continuously growing record of transactions, copied across many independent computers, in which each new block of data is cryptographically linked to the block that came before it. Two ideas in that definition do almost all of the work. First, each block stores the hash—a short cryptographic fingerprint—of the previous block, which is the literal “chain” in “blockchain.” Second, no single company or government owns the ledger; the network as a whole maintains it collectively. Everything else covered in this module is really an elaboration of those two ideas: how the fingerprinting works, how thousands of independent computers agree on what to fingerprint, and what becomes possible once you have a tamper-evident, jointly maintained record that nobody unilaterally controls.

It is worth pausing on why this matter. Every financial and administrative system before blockchain relied on a trusted intermediary—a bank to certify that money moved correctly, a land registry office to certify that a property changed hands, a notary to certify that a document is genuine. Blockchain does not eliminate the need for verification; it relocates that verification from

a single trusted party to a mathematical and economic process that thousands of independent parties can each check for themselves.

1.1 History and Evolution

Blockchain's underlying ideas did not appear fully formed in 2008. They were assembled over almost two decades from separate strands of cryptography and distributed-systems research.

1991 Haber and Stornetta. Stuart Haber and W. Scott Stornetta proposed a cryptographically linked chain of blocks to timestamp digital documents. Their method used cryptographic hash functions to link records together, so that any attempt to alter a previously stored document would be immediately detectable. Although their work had nothing to do with digital currency, it established the core concept later inherited by blockchain: an immutable, tamper-evident ledger built from linked cryptographic fingerprints.

1998 Nick Szabo's "Bit Gold." Szabo proposed a theoretical decentralized digital currency that did not depend on banks or governments. The scheme envisioned cryptographic puzzles that created provably scarce digital assets, verified by a distributed network rather than a central authority. Bit Gold was never implemented, but its combination of decentralization, digital scarcity, and cryptographic security directly foreshadowed Bitcoin. The analogy Szabo had in mind is straightforward: gold is valuable partly because it is difficult to mine and cannot be manufactured at will; Bit Gold proposed the same property for a digital asset.

2008 the Bitcoin white paper. In October 2008, a person or group writing under the pseudonym Satoshi Nakamoto published "Bitcoin: A Peer-to-Peer Electronic Cash System." The paper described a decentralized payment system that allows two parties to transact securely without a trusted third party such as a bank. Crucially, it also solved a problem earlier proposals had not fully solved: how to prevent the same digital coin from being spent twice without a central ledger-keeper. Its answer combined cryptographic hashing, a distributed network, and a competitive puzzle-solving process Proof of Work into a single working design.

2009 the Genesis Block. On 3 January 2009, Nakamoto mined Bitcoin's first block, known as Block 0 or the Genesis Block. It contained a reference to a newspaper headline about the global financial crisis, a detail widely read as a statement of purpose: blockchain was conceived, in part,

as an alternative to a financial system that had just shown its fragility. The successful mining of the Genesis Block turned blockchain from a theoretical proposal into an operating, decentralized, tamper-evident ledger.

2015 Ethereum launches. Vitalik Buterin and collaborators launched Ethereum, which introduced a Turing-complete blockchain—one capable of running arbitrary programs, not just recording payments. This single change made smart contracts and decentralized applications (dApps) possible, and it is the reason blockchain expanded far beyond digital cash.

2017–2018 the ICO boom, sometimes called “Blockchain 2.0.” Initial Coin Offerings let new projects raise capital by selling tokens directly to investors, bypassing traditional venture funding. Thousands of projects launched, attracting billions of dollars. Many ICOs failed or were outright fraudulent, but the boom also proved that token-based fundraising could work at scale and accelerated the growth of the wider ecosystem. (Chapter 9 covers ICOs, and the regulatory backlash they triggered, in depth.)

2019 onward DeFi, NFTs, and DAOs, sometimes called “Blockchain 3.0.” Decentralized Finance (DeFi) let users borrow, lend, trade, and earn interest without a bank. Non-Fungible Tokens (NFTs) introduced verifiable ownership of unique digital assets such as artwork and collectibles. Decentralized Autonomous Organizations (DAOs) let communities govern shared projects through on-chain voting rather than a conventional management hierarchy.

2022 Ethereum’s “The Merge.” In September 2022, Ethereum completed a major upgrade that replaced its energy-intensive Proof of Work consensus mechanism with the far more efficient Proof of Stake model, cutting the network’s energy consumption by more than 99% while preserving its security and decentralization. Chapter 4 explains exactly how both mechanisms work and why this trade-off exists.

Worked example: why anyone would bother with an ICO

Imagine a startup wants to build a decentralized ride-sharing platform—similar in spirit to Uber, but without a central company controlling it. Rather than approaching a bank for a loan or selling shares to venture capitalists, the startup issues a digital token, say RideToken (RIDE), and publishes a white paper explaining how the token will be used. Investors purchase RIDE using

Bitcoin or Ether, and the startup raises the capital it needs to build the platform. Once launched, passengers pay for trips in RIDE and drivers are paid in RIDE; if the platform succeeds, demand for the token and therefore its value tends to rise with usage. The appeal for the startup is that it obtains funding without going through a bank or a venture capital firm, while investors receive an asset they can use on the platform or trade on an exchange. As you will see in Chapter 9, this same mechanism also created enormous room for fraud and speculation, which is why ICOs attracted intense regulatory scrutiny.

1.2 Key Terminology

Precise vocabulary matters in this field, because loosely related words like “storage” and “ledger,” or “distributed” and “decentralized” are often confused, and the confusion tends to compound as the module progresses.

Block. A bundle of transactions plus metadata: a timestamp, the previous block’s hash, and a summary hash of the block’s own contents (the Merkle root, introduced in Chapter 2).

Chain. The sequence of blocks, each one referencing the hash of the block before it, which creates a tamper-evident history: change one block, and every hash that follows becomes invalid.

Node. A computer participating in the network by storing and/or validating the ledger. Some nodes store the complete history (full nodes); others perform lighter-weight tasks.

Ledger. The complete, continuously updated record of every transaction that has ever occurred on the network. It is worth carefully distinguishing a ledger from storage in general. Storage is simply where data is kept—a hard disk, an SSD, a cloud server—and it can hold any kind of data at all: documents, photos, videos. A ledger, by contrast, is a structured record specifically of transactions, designed to be permanent (or at least tamper-evident) rather than freely editable. Every blockchain node stores a copy of the ledger on its own storage device, but the ledger itself is the organized record of transactions, not the physical medium it happens to sit on. A useful analogy: a filing cabinet is the storage; the accounting book inside it is the ledger.

Decentralization. Distributing control away from a single authority and across a distributed network of independent participants.

Hash function. A mathematical function that converts any input into a fixed-size output; it is the security backbone of blockchain and is covered in full in Chapter 2.

Consensus mechanism. The process by which the network agrees on which transactions and which blocks are valid, without a trusted central party. Chapter 4 is devoted entirely to this topic.

Wallet. Software or hardware that stores the cryptographic keys used to access and transact digital assets. Importantly, a wallet does not “hold” coins the way a physical wallet holds cash; it holds the keys that prove ownership of coins recorded on the blockchain.

Mining, in brief

Because it appears throughout this chapter’s discussion of Bitcoin’s history, it is worth previewing one more term here, even though Chapter 4 covers it in full: mining is the process by which computers compete to solve cryptographic puzzles in order to verify new transactions, add a new block to the blockchain, and receive newly created cryptocurrency plus transaction fees as a reward. Mining therefore serves two purposes simultaneously—it secures the blockchain, and it is how new cryptocurrency is issued.

Digital scarcity

Digital scarcity is the principle that a digital asset has a strictly limited supply, which is what makes it valuable despite being infinitely copyable in principle. An ordinary computer file—a Word document, a photograph—can be duplicated without limit, which is precisely why files by themselves have no scarcity-based value. Bitcoin is different: the protocol enforces a maximum supply of 21 million coins, and the blockchain ensures that each coin is owned by exactly one address at a time. NFTs apply the same principle to individual digital items: each one is unique, and ownership is recorded on-chain, so—unlike a JPEG file, which can be copied infinitely—an NFT cannot be silently duplicated.

1.3 Principles of Decentralized Systems: Four Pillars

Distributed control. No single authority—no person, company, or government—controls the blockchain network. Instead, thousands of independent nodes collectively verify transactions, maintain the ledger, and enforce the network’s rules. This removes any single point of failure, making the system harder to shut down, censor, or quietly manipulate. No single node can change

the ledger without the rest of the network's agreement, and that agreement is enforced by the consensus mechanism.

Transparency. On a public blockchain, every transaction is visible to every participant. Anyone can verify a transaction, trace the movement of an asset, or audit the ledger's entire history. Participants' real-world identities may remain pseudonymous, but the underlying data is openly auditable by anyone who cares to look. This stands in sharp contrast to a bank's internal database, which is controlled and viewable only by the bank itself.

Immutability. Once data has been recorded and sufficiently confirmed on the blockchain, it cannot be changed, deleted, or overwritten. Each block is cryptographically linked to the one before it, so altering a single past block would require recalculating every subsequent block's hash and then outpacing the honest network doing so—a task that is, on any sufficiently large network, computationally infeasible. This gives blockchain a permanent, tamper-evident record, but it is a double-edged property: the same immutability that prevents fraud also means genuine mistakes sending funds to the wrong address, for instance—are very hard to reverse. Correcting an error requires adding a new transaction; it cannot involve editing the old one.

Trustlessness. “Trustless” is a term that confuses many newcomers, because it does not mean “no trust is involved.” It means that users do not need to trust each other, or any single counterparty, in order to transact safely. Trust is instead established through cryptographic techniques, consensus rules, and economic incentives that make honest behavior the rational choice. You are not trusting a stranger's word; you are trusting the mathematics and the incentive structure that make cheating unprofitable.

A convenient way to hold all four ideas at once: identity (“who are you?”) is answered by your blockchain address; authorization (“do you have permission?”) is answered by your private key; and integrity (“has the data remained unchanged?”) is answered by hash functions detecting any modification.

1.4 A Common Point of Confusion: Centralized vs. Distributed vs. Decentralized

These three words are frequently used as if they meant the same thing. They do not, and the distinction is one of the most consequential ideas in this chapter.

Centralized describes who controls a system, not where its hardware sits. A single authority controls data and rules; the infrastructure may or may not be physically distributed. Consider a bank with branches in several cities, each running local servers, plus backup data centers elsewhere. The infrastructure is physically spread out, but every branch answers to the same headquarters: if headquarters decides to freeze an account, every branch complies. The key point is that the infrastructure may be spread across many locations, while control remains firmly centralized.

Distributed describes physical or logical layout, not authority and a distributed system can still be entirely centrally controlled. Netflix and Google Cloud both operate servers in data centers scattered around the world, so that content is delivered from whichever server is closest to a given user. Even so, one organization alone decides what content exists, controls user accounts, and can add or remove anything at any time. The resources are distributed; control is not.

Decentralized means authority is genuinely shared across independent nodes, and it happens to be both distributed and decentralized at once. The Bitcoin network consists of thousands of independent nodes worldwide, each storing a full copy of the ledger and independently verifying transactions. No company, government, or individual owns or controls the network; protocol changes require broad agreement from the community rather than approval from a single authority, and if any individual node goes offline, the rest of the network continues operating without interruption.

Concept	Real-World Example	What It Demonstrates
Centralized	A bank with branches nationwide	Data may exist in many places, but one headquarters controls everything
Distributed	Netflix or Google Cloud	Servers are spread worldwide, but one company controls the entire system
Decentralized	Bitcoin or Ethereum	Nodes are spread worldwide, and no single entity controls the network or its rules

Chapter 1 Recap: Key Takeaways

- Blockchain evolved over decades from 1990s cryptographic timestamping research, through Bitcoin's 2008–2009 launch, to today's programmable, multi-purpose networks.
- Precise vocabulary matters: block, chain, node, ledger, hash function, and consensus mechanism are the building blocks for everything that follows in this module.
- Four principles distributed control, transparency, immutability, and trustlessness define what makes a blockchain meaningfully different from an ordinary database.
- “Distributed” describes physical or logical layout; “decentralized” describes where authority sits. Public blockchains like Bitcoin and Ethereum are both distributed and decentralized at once, which is why they are often used as the canonical examples of each term.

Discussion Questions and Model Answers

1. Why is the 2008 Bitcoin white paper considered a turning point rather than the absolute starting point of blockchain technology?

The Bitcoin white paper is a turning point rather than an origin point because it combined ideas that already existed cryptographic hashing, distributed systems, digital signatures, and the concept of a decentralized digital currency into a practical, working design for a blockchain-based payment system. The theoretical groundwork had already been laid by researchers such as Haber and Stornetta in 1991 and Szabo in 1998. Bitcoin was the first implementation that demonstrated these ideas could actually function together, at scale, in a real decentralized network.

2. Give one real-world example each of a distributed system and a decentralized system that were not already discussed in the chapter.

A Content Delivery Network such as Cloudflare is a good example of a distributed-but-centralized system: Cloudflare operates servers in hundreds of geographic locations to deliver content quickly, but Cloudflare alone owns and controls the entire infrastructure distributed, but centrally controlled. Mastodon is a good example of a genuinely decentralized system: it is a social media platform made up of many independently operated servers (“instances”), each managed by a different administrator, with no single company controlling the network as a whole both distributed and decentralized.

3. Why is immutability described as both a strength and a limitation of blockchain systems?

Immutability is a strength because it prevents confirmed transactions from being altered or deleted, which protects data integrity, supports transparency, and guards against fraud. It is simultaneously a limitation because genuine mistakes – sending cryptocurrency to the wrong address, or recording incorrect information – cannot be corrected or removed after the fact. The only way to fix an error is to add a brand-new transaction; the original, erroneous record remains permanently on the blockchain.

Chapter 2: Cryptographic Foundations

Hash Functions · Public-Key Cryptography · Digital Signatures

Learning Objectives

By the end of this chapter, you should be able to:

- Explain how cryptographic hash functions work, and describe the three properties deterministic, avalanche effect, pre-image resistant that make them useful for blockchain.
- Explain how a Merkle tree summarizes an entire block's transactions into one root hash, and why this makes verification efficient.
- Explain the relationship between a private key, a public key, and a blockchain address.
- Describe how digital signatures provide authentication, integrity, and non-repudiation.
- Walk through the full worked example of a signed transaction, from creation to network-wide verification.

2.0 Setting the Scene: Why Does Blockchain Need Cryptography?

Blockchains operate on an open, adversarial network with no central authority to check identification or approve transactions. Cryptography replaces that missing authority with mathematical guarantees, and it does so through three tools working together: hash functions, key pairs, and digital signatures. Each tool answers a different question.

Integrity has the data remained unchanged? Cryptographic hash functions ensure that if even a single character of a transaction or block is altered, its hash changes completely, making tampering immediately detectable. Suppose Alice sends 2 BTC to Bob, and the transaction is recorded with its corresponding hash. If an attacker tries to change the recorded amount to 20 BTC, the hash of the transaction changes too, the block becomes invalid, and the network rejects the modification. Integrity, in short, ensures that blockchain data cannot be secretly altered.

Identity who owns this account or asset? Instead of usernames or national identification numbers, blockchain identifies participants through public/private key pairs. The public key (or the blockchain address derived from it) represents a user's identity; the private key proves that the user actually owns that identity. When Bob creates a wallet, it generates a private key that he keeps

secret, a corresponding public key, and a blockchain address derived from that public key. When Alice wants to send Bob cryptocurrency, she sends it to his address, which uniquely identifies him on the network.

Authorization do you have permission to do this? Authorization is granted by digitally signing a transaction with a private key. If Bob owns 5 ETH and wants to send 1 ETH to Carol, he creates the transaction and signs it with his private key; the network then verifies that signature using Bob's public key. If the signature checks out, the transaction is authorized and added to the blockchain. If anyone else tries to move Bob's funds without his private key, the transaction is rejected outright.

Concept	Meaning	Blockchain Use Case
Integrity	Ensures data remains accurate and unchanged	A transaction altered from 2 BTC to 20 BTC is detected and rejected
Identity	Uniquely identifies users via public/private key pairs	Bob receives cryptocurrency through his blockchain address
Authorization	Grants permission to perform an action	Bob signs a transaction with his private key to send 1 ETH to Carol

2.1 Cryptographic Hash Functions

A hash function takes an input of any size and produces a fixed-size output, called a digest. Bitcoin uses SHA-256, which produces a 256-bit output; Ethereum uses Keccak-256. No matter how large the input a single character or an entire book the output is always the same fixed length.

Three properties make hash functions suitable for blockchain:

Deterministic. The same input always produces exactly the same output. This predictability is essential: if hashing were not deterministic, no two nodes could ever agree that a given piece of data hashes to a given value.

Avalanche effect. Changing even a single character of the input produces a completely different output. This is what makes tampering immediately visible: there is no such thing as a “slightly different” hash for a “slightly different” transaction.

Pre-image resistant. It is computationally infeasible to reverse a hash output back into its original input. You can verify that a given input produces a given hash, but you cannot work backward from the hash to reconstruct the input.

A fourth property, collision resistance – it must be infeasible to find two different inputs that produce the same hash – underpins all three of the blockchain applications described next.

How blockchains actually use hashing

Linking blocks. Each block header stores the hash of the previous block. This is the literal “chain” in blockchain: change any piece of data in a past block, and every hash computed from that point forward changes as well, immediately breaking the chain and revealing the tampering.

Merkle trees. All of the transactions inside a single block are summarized into one Merkle root, which allows fast, lightweight verification without needing to examine every individual transaction. A Merkle tree is a hierarchical structure in which pairs of transaction hashes are repeatedly combined and re-hashed until a single hash – the Merkle root – remains. That root is stored in the block header and uniquely represents every transaction in the block.

Worked example. Suppose a block contains four transactions: T1 (Alice → Bob, \$10), T2 (Bob → Carol, \$5), T3 (Carol → David, \$8), and T4 (David → Eve, \$12). Step one hashes each transaction individually, producing H1, H2, H3, and H4. Step two combines and hashes pairs: $H_{12} = \text{Hash}(H1 + H2)$ and $H_{34} = \text{Hash}(H3 + H4)$. Step three combines the two results into the Merkle Root = $\text{Hash}(H_{12} + H_{34})$. The practical payoff is efficiency: to verify that Transaction #500 belongs to a block of 1,000 transactions, you do not need to download and hash all 1,000 transactions. You only need Transaction #500 itself, a handful of related hashes along its path to the root (the “Merkle proof”), and the Merkle root already stored in the block header. This is especially valuable for lightweight wallets and resource-constrained devices that cannot store the entire blockchain.

Proof of Work mining. Miners search for a nonce whose resulting hash meets a difficulty target. Chapter 4 covers this process in full detail; for now, the key point is simply that it is another application of the same hash function properties described above.

2.2 Public-Key Cryptography

Public-key cryptography uses a pair of mathematically related keys. The **private key** is a secret, randomly generated number that must never be shared; it proves ownership and authorizes spending. The **public key** is mathematically derived from the private key, can be shared openly, and is hashed to form a wallet address. Bitcoin and Ethereum both use the secp256k1 elliptic curve, via the ECDSA algorithm, to generate these key pairs. Deriving the public key from the private key is computationally easy; deriving the private key from the public key is, for all practical purposes, impossible.

From public key to address

A public key can be safely shared with anyone; it is mathematically transformed (hashed) into a blockchain address, functioning much like a bank account number that others can use to send you funds. Suppose Alice wants to send Bob 1 BTC. Bob generates a key pair, derives a public address such as bc1qxy2kgdygjrsqtzq2n0yrf2493p83kkfjhx0wlh, and shares that address with Alice. Alice sends the Bitcoin to that address, and only Bob because only Bob knows the corresponding private key can subsequently spend it. Sharing a public key or address does not give anyone the ability to withdraw funds; it only allows them to send funds to you, in the same way that sharing your bank account number lets people deposit money but not withdraw it.

Secure identities and ownership

In blockchain, identity is represented by a public key or address, while ownership of an asset is proven by possessing the corresponding private key. Suppose Carol owns an NFT recorded on the blockchain as belonging to her address. To sell it to David, Carol signs the transfer transaction with her private key; the blockchain verifies that signature using her public key, and ownership transfers to David once the verification succeeds. No one else can sell Carol's NFT, because no one else possesses her private key.

No private key, no recovery

Whoever holds a private key has complete, irrevocable control over the associated funds and this control cuts both ways. In traditional banking, a forgotten password can be reset and a lost card reissued, because a trusted institution stands ready to restore access. On a blockchain, losing a

private key with no backup means the associated funds are permanently inaccessible; there is no customer service line, no manager, and no institution that can help. This is arguably the single most consequential practical difference between blockchain-based finance and traditional banking, and it is why private key management (hardware wallets, seed phrase backups, multi-signature schemes) is treated as such a serious topic in practice.

2.3 Digital Signatures

A digital signature is cryptographic proof that a transaction was created by the legitimate owner of a private key. The process unfolds in three steps. First, the transaction details are run through a hash function. Second, the sender applies their private key to that hash, producing a signature. Third, anyone can check that signature against the sender's public key to confirm it is genuine. If even one character of the transaction data changes after signing, verification fails immediately this is the avalanche effect from Section 2.1 doing its work at the application layer.

Digital signatures provide three simultaneous guarantees:

Authentication. The recipient can be confident the transaction genuinely came from the claimed sender, because the network verifies Alice's signature using her public key, proving the transaction was created by Alice and not by an impostor.

Non-repudiation. Because only the private key holder could have produced a valid signature, the sender cannot credibly deny having authorized the transaction after the fact. Once a transaction is confirmed, Alice cannot later claim she never sent it.

Integrity. Any change to the signed data, however small, causes the signature verification to fail. If an attacker changes a transaction amount from 2 BTC to 20 BTC after Alice signed it, the signature no longer matches, and the blockchain rejects the modified transaction.

Every node independently verifies all three guarantees before accepting a transaction no central authority is required to vouch for any of it.

2.4 Putting It Together

The three tools introduced in this chapter are complementary, not redundant. Hash functions protect the integrity of data they make tampering detectable. Public-key cryptography establishes

secure identities and ownership – it answers who is who, without a central registry. Digital signatures verify that only an authorized user can approve a transaction – they turn a private key into proof of intent. Together, these three tools let a blockchain provide a secure, transparent, tamper-resistant, decentralized system without relying on a trusted central authority.

It is worth being precise about what cryptography does not do on its own. Cryptography secures data integrity and authenticity for an individual transaction, but it says nothing about the order in which transactions should be accepted across the whole network, or about which of two conflicting transactions should win. That question – how a decentralized network agrees on one shared transaction history – requires a consensus mechanism, which is the subject of Chapter 4.

Worked Example: Carla Sends a Transaction

1. Carla’s wallet builds the transaction: sender, recipient, amount, and a nonce (a number used to prevent replaying the same transaction twice).
2. The wallet computes the hash of this transaction data using SHA-256 (or Keccak-256 on Ethereum).
3. Carla’s private key signs that hash, producing a signature made of two large numbers, conventionally called *r* and *s*.
4. The signed transaction is broadcast to the network. Every node independently verifies the signature against Carla’s public key before accepting it.
5. If anyone had altered even one digit of the amount after Carla signed it, the hash – and therefore the signature check – would fail instantly, and the tampered transaction would be rejected across the network.

Chapter 2 Recap: Key Takeaways

- Hash functions are deterministic, exhibit the avalanche effect, and are practically irreversible, which makes them ideal for linking blocks together and for building Merkle trees.
- Public-key cryptography gives every user a private key (kept secret) and a public key (shared openly), mathematically linked so that deriving the private key from the public key is effectively impossible.

- Digital signatures provide authentication, non-repudiation, and integrity all at once, letting nodes verify transactions without any central authority.
- Together, these tools replace institutional trust with mathematical guarantees but network-wide agreement on transaction ordering still requires a consensus mechanism, covered next in Chapter 4.

Discussion Questions and Model Answers

1. If a user's private key is compromised, what are the practical consequences, and why is recovery so much harder than with a bank account?

Whoever possesses a compromised private key can authorize transactions and transfer the associated cryptocurrency without permission, and because blockchain transactions are irreversible, stolen funds usually cannot be recovered. Recovery is much harder than with a bank account because there is no central authority no bank that can freeze the account, reverse the fraudulent transactions, or issue the user a new private key. Protecting a private key is therefore the user's sole responsibility, with no institutional safety net.

2. Explain the three security guarantees of digital signatures using a concrete cryptocurrency payment example.

Suppose Alice sends 2 BTC to Bob by signing the transaction with her private key. Authentication means the network verifies Alice's digital signature using her public key, confirming the transaction was created by Alice and not by an impostor. Integrity means that if an attacker changes the transaction amount from 2 BTC to 20 BTC, the signature becomes invalid and the blockchain detects and rejects the change. Non-repudiation means that once the transaction is confirmed, Alice cannot later deny sending it, because the valid signature proves it was authorized using her private key alone.

3. Why is a Merkle tree more efficient than hashing an entire block's transaction list as one long string?

A Merkle tree lets the network verify an individual transaction without processing every transaction in the block. Instead of re-hashing the full transaction list every time verification is needed, only the hashes lying along the path from the target transaction up to the Merkle root are

required the Merkle proof. This dramatically reduces the computation, bandwidth, and storage needed for verification. To verify Transaction #500 in a block of 1,000 transactions, a lightweight wallet needs only that transaction, a small Merkle proof, and the Merkle root already stored in the block header, rather than the other 999 transactions.

Chapter 3: How Blockchain Works: A Transaction from Creation to Verification

Learning Objectives

By the end of this chapter, you should be able to:

- List and explain the six stages a transaction passes through, from creation to becoming a permanent part of the blockchain.
- Explain why nodes verify transactions independently rather than trusting whoever broadcasts them.
- Explain how blocks link together through hashes to form a tamper-evident ledger.
- Connect the four key benefits of blockchain – decentralized, secure, transparent, immutable – back to the mechanics that produce them.

3.0 Setting the Scene: Why Does a Transaction Need Verification?

On a blockchain, there is no bank teller and no clerk who approves a payment. Nodes scattered across the world must independently agree that a transaction is genuine before it becomes a permanent part of the record. Three questions have to be answered for every transaction:

Identity. Was the transaction created by the legitimate owner of the digital assets involved, proven through their private key and digital signature, rather than by an impostor?

Validity. Does the transaction follow the blockchain’s rules – does the sender actually have sufficient funds, is the digital signature correct, and have the assets in question not already been spent (preventing so-called “double spending”)?

Permanence. Once verified and added to a block, does the transaction become a permanent part of the blockchain that cannot be altered or deleted without invalidating the entire chain that follows it?

Six steps carry a transaction from creation to permanent record, and each one removes the need for a trusted middleman that a traditional financial system would otherwise require.

3.1 The Transaction Lifecycle, Part 1: From Creation to Network Verification

Step 1 Transaction created. A user, call her A, wants to send a digital asset to another user, B. A's wallet builds the transaction details: sender, recipient, amount, and supporting data such as a nonce and a digital signature.

Step 2 Transaction broadcast. The transaction is broadcast to the peer-to-peer network of nodes, spreading rapidly from node to node across the participants who make up the network.

Step 3 Verification by nodes. Nodes independently verify the transaction against the network's agreed rules. This verification step is where the consensus mechanism covered fully in Chapter 4 actually does its work: nodes are not simply trusting each other's word, they are each independently applying the same rules to the same data.

3.2 The Transaction Lifecycle, Part 2: From Block Creation to Permanent Record

Step 4 Block created. Once verified, the transaction is grouped together with other pending transactions to form a candidate block, which consists of a header, the transactions themselves, and a unique hash identifying the block.

Step 5 Block added to the chain. The block is appended to the blockchain. Once attached, it is treated as permanent and tamper-evident.

Step 6 Transaction complete. The transaction is now complete and recorded identically across every full node on the network.

3.3 The Distributed Ledger: Blocks Linked by Hashes

Consider three consecutive blocks. Block 101 is timestamped at 10:15 AM, has a previous hash of 0000...00 (indicating it follows the genesis block or an earlier ancestor), and its own hash is 6A3F...9P7B. Block 102, timestamped two minutes later at 10:17 AM, stores 6A3F...9P7B as Block 101's hash as its "previous hash" field, and computes its own hash as 8B7C...4D2E. Block 103, timestamped at 10:20 AM, stores 8B7C...4D2E as its previous hash and computes F2D4...7G8H as its own. Each block also contains its own set of transactions (Tx1, Tx2, Tx3, and so on for Block 101; Tx4, Tx5, Tx6 for Block 102; and so forth).

The mechanism that makes this tamper-evident is simple to state and powerful in effect: each block stores the hash of the previous block. Change any data anywhere in that history, and every hash computed from that point forward changes as well — this is exactly the avalanche effect from Chapter 2 propagating through the whole chain, not just a single block. An attacker cannot quietly edit Block 101 without also having to recompute Block 102’s hash, then Block 103’s hash, and so on for every subsequent block, all while the rest of the honest network keeps extending the chain further ahead of them.

3.4 Key Benefits: What This Process Guarantees

Decentralized. No single entity controls the data; every full node holds its own copy of the ledger.

Secure. Cryptographic hashing combined with consensus rules protects the data from tampering.

Transparent. All participants can independently verify any transaction on the chain for themselves.

Immutable. Once a block has been added and sufficiently confirmed, its data cannot be changed or deleted.

Worked Example: Alice Sends Bob 0.5 BTC

1. Alice’s wallet builds the transaction: sender, recipient, amount, and a nonce.
2. The transaction is broadcast to the peer-to-peer network of nodes.
3. Nodes verify Alice’s signature and confirm she has sufficient funds.
4. Miners group the now-valid transaction together with others into a new block and add it to the chain.
5. The block propagates across the network, and Bob’s wallet now shows the funds as received.

Chapter 3 Recap: Key Takeaways

- A transaction passes through six stages: creation, broadcast, verification, block creation, chain addition, and completion.
- Nodes verify transactions against agreed-upon rules before accepting them; no central authority is needed at any step.

- Each block stores the hash of the block before it, so altering any past data breaks every hash that follows it.
- The end result is a ledger that is simultaneously decentralized, secure, transparent, and immutable properties that all participants can trust without trusting each other.

Discussion Questions and Model Answers

1. Why is it important that nodes verify a transaction independently rather than trusting the broadcaster?

Independent verification ensures that only valid, legitimate transactions are ever added to the blockchain. If nodes simply trusted whoever broadcast a transaction, a malicious user could submit fraudulent transactions for instance, spending funds they do not actually own, or attempting to spend the same funds twice (double spending) and there would be no independent check to catch it.

2. Explain how a single altered transaction in Block 102 would be detected by the rest of the network.

Changing a transaction inside Block 102 changes Block 102's hash. Because Block 103 stores the hash of Block 102 in its own header, the two no longer match once Block 102 is altered, which breaks the chain at that point. Other nodes, each independently checking that stored hashes match, detect this mismatch and reject the altered version of the blockchain as invalid.

3. Which of the four key benefits decentralized, secure, transparent, immutable would be hardest to achieve with a traditional centralized database, and why?

Immutability would be the hardest to achieve, because the administrators of a centralized database retain the technical ability to modify or delete records at will; nothing in the architecture of a typical database prevents this. Blockchain, by contrast, links every block to the next through cryptographic hashes, which makes recorded data practically impossible to alter without the alteration being immediately detected by the rest of the network a property a conventional, centrally administered database simply does not have by default.

Chapter 4: Consensus Mechanisms and Mining

How Decentralized Networks Compete, Agree, and Stay Secure

Learning Objectives

By the end of this chapter, you should be able to:

- Explain the Byzantine Generals Problem and why it is central to the design of any consensus mechanism.
- Walk through the full mechanics of Proof of Work mining: the mempool, candidate blocks, the nonce search, the difficulty target, and difficulty adjustment.
- Explain the three components of miner compensation: the block reward, transaction fees, and the halving schedule.
- Explain how Proof of Stake works, including staking, validator selection, and slashing, and why Ethereum adopted it.
- Describe Delegated Proof of Stake, Practical Byzantine Fault Tolerance, and Proof of Authority, and the centralization/speed trade-off they represent.
- State the blockchain trilemma and use it to compare consensus mechanisms.

4.0 The Problem Consensus Mechanisms Solve: Agreement Without a Referee

In a decentralized network, multiple nodes might propose conflicting versions of the next block at the same moment. With no central authority, how does the network agree on which one is correct? This is called agreement without a referee. In a traditional system, a trusted central authority – a bank, for instance – simply decides which transactions are valid. A blockchain has no bank, no administrator, and no central server; thousands of independent nodes must instead reach agreement on the very same version of the blockchain on their own.

Think of a football match. Normally, a referee decides whether a goal counts or whether a foul occurred. Now imagine the same match with no referee at all: the players themselves must agree on every call. Blockchain works similarly – there is no central referee, so nodes must agree by all following the same publicly known consensus rules.

At any given moment, several miners or validators might construct different candidate blocks from the same pool of pending transactions. The blockchain cannot add all of them; the network must settle on exactly one. Depending on the blockchain in question, the specific mechanism used to settle this differs – Bitcoin uses Proof of Work, Ethereum uses Proof of Stake – but the goal is the same: allow every honest node to converge on the same next block without anyone acting as a central referee.

The Byzantine Generals Problem

This is a classic problem in computer science: how can multiple computers agree on the correct decision if some participants may be faulty or actively malicious? Imagine four military generals surrounding a city, all of whom must reach the same decision – attack together, or retreat together. If even one dishonest general sends contradictory messages to different peers, the honest generals might end up disagreeing and the coordinated action fails. Blockchain consensus mechanisms are designed to solve exactly this problem: they let honest participants reach the same decision even when some participants behave maliciously.

Consensus mechanisms accomplish this by setting transparent rules – for instance, in Bitcoin, “the first miner whose hash satisfies the difficulty target wins” – that every participant knows in advance and that grant no one special treatment. They also make dishonest behavior more expensive than honest participation. In Proof of Work, an attacker who wants to alter a confirmed transaction would need to control more than 50% of the network’s total hash power, and would need to purchase enormous amounts of mining hardware and pay enormous electricity costs to do so – a task made deliberately, and increasingly, uneconomical as a network grows.

Part 1: Mining and Proof of Work – Bitcoin’s Original Security Model

4.1 Who Decides Which Node Proposes the Next Block?

Thousands of nodes each hold a copy of the ledger, but only one of them gets to propose the next block at any given moment. Mining is the competition that decides who. Proof of Work makes that competition costly to win but cheap to verify, so that no single party can cheaply seize control of the chain. Three ideas are in tension here: computation (the competition itself requires real computational work), fairness (the puzzle gives no participant a built-in advantage beyond raw

computing power), and security (the resulting asymmetry between the cost of winning and the cost of verifying is what protects the network).

4.2 What Is Mining? Competing to Propose the Next Block

Miners are specialized nodes that collect pending transactions and race to assemble the next valid block. The puzzle requires miners to repeatedly hash the candidate block header with a changing nonce, searching for an output below a target value. The reward goes to the first miner to find a valid hash: they broadcast the block and claim the block reward plus transaction fees.

Mining is deliberately hard to win but trivial for any other node to check – this asymmetry is what secures the network. Suppose the network’s difficulty target requires a hash beginning with six leading zeros. A miner starts with nonce = 0, hashes the block header, and gets an invalid result; tries nonce = 1, gets another invalid result; and continues this way, potentially for millions or billions of attempts, until – say, at nonce = 4,587,231 – the resulting hash happens to satisfy the target. Finding that nonce might have taken millions of attempts. Verifying it, once broadcast, takes exactly one hash calculation: another node simply hashes the same block header with the winning nonce and checks whether the result falls below the difficulty target. If it does, the block is accepted – a single computation, versus millions or billions on the miner’s side.

4.3 The Full Mining Process, Step by Step

1. A transaction enters the network. Suppose Alice sends 2 BTC to Bob. The transaction contains a sender address, a receiver address, an amount, and a digital signature, and Alice’s wallet broadcasts it to the network.

2. Verification. Every full node independently checks that the digital signature is valid, that Alice actually owns the Bitcoin she is trying to spend, and that it has not already been spent.

3. The mempool. A verified but not-yet-confirmed transaction sits in the mempool – a temporary storage area held in the RAM of every full node – alongside other pending transactions, each node maintaining its own local copy.

4. Selecting transactions. Each miner monitors its own local mempool and, when preparing a new block, selects transactions to include – typically prioritizing those offering the highest transaction fees, since those fees add to the miner’s eventual reward.

5. Building a candidate block. The miner assembles the selected transactions into a candidate block containing the previous block's hash, the selected transactions, a Merkle root, a timestamp, the current difficulty target, and a nonce. This candidate block is not yet part of the blockchain.

6. The nonce search. A nonce ("Number Used Once") is a 32-bit value in the block header whose only purpose is to change the hash of the block header. It has no independent mathematical meaning. Mining software starts with nonce = 0; if the resulting hash of the full header does not meet the target, it tries nonce = 1, then 2, then 3, and so on. Because the block header (previous hash, Merkle root, timestamp, difficulty target, and nonce, all combined) is passed through SHA-256, and because SHA-256 exhibits the avalanche effect, changing only the nonce by one still produces a completely different hash each time.

7. The difficulty target. Bitcoin defines a large number called the difficulty target; a block is accepted only if its generated hash, read as a number, is numerically smaller than that target. Because SHA-256 produces a 256-bit hexadecimal number, and smaller hexadecimal numbers naturally have more leading zeros, miners commonly describe this informally as "finding a hash with enough leading zeros" but the precise rule is that the hash's numeric value must fall below the target.

8. Difficulty adjustment. Bitcoin automatically re-targets its difficulty every 2,016 blocks, roughly every two weeks, with the goal of keeping the average block time close to ten minutes. If blocks are being found too quickly, difficulty rises; if too slowly, it falls.

9. Broadcasting and verifying the winning block. Once a miner finds a valid hash, the candidate block becomes an official valid block and is broadcast to the network. Every other node independently verifies the Proof of Work, every transaction inside the block, the previous block hash, and the overall block structure before appending it to their own copy of the chain.

10. Clearing the mempool. Once a block is accepted, every transaction it contains is removed from each node's local mempool, since those transactions are now permanently recorded on the blockchain.

4.4 Rewards and Incentives

Block reward. Newly created coins paid to the miner who successfully finds a valid block. When Bitcoin launched in 2009, the reward was 50 BTC per block – deliberately generous, since very few people were mining at the time and the network needed to attract participants to help secure it.

Transaction fees. Senders can attach a fee to their transaction to encourage miners to include it sooner. Because block space is limited, miners typically prioritize the highest-fee transactions first, which is why fees tend to rise during periods of network congestion – if a mempool holds 20,000 pending transactions but a block can include only 3,000, the transactions offering the highest fees are the ones most likely to be included next.

The halving. Roughly every four years (every 210,000 blocks), Bitcoin’s block reward is cut in half: from 50 BTC in 2009, to 25 in 2012, 12.5 in 2016, 6.25 in 2020, and 3.125 in 2024. This scheduled reduction caps Bitcoin’s total supply at 21 million coins and gradually shifts miner revenue away from block rewards and toward transaction fees over time. After each halving, mining a block becomes less profitable in BTC terms unless the price of Bitcoin rises or fees make up a larger share of the total reward – for example, a miner earning 6.25 BTC plus 0.20 BTC in fees before the 2024 halving would earn only 3.125 BTC plus 0.20 BTC in fees afterward, with the fee component unchanged but the block-reward component cut in half.

4.5 Security Strength, Energy Cost

Battle-tested security. Proof of Work has protected Bitcoin from attack for well over a decade. To alter a confirmed block, an attacker must redo the Proof of Work for that block and for every block mined after it, all while mining faster than the entire honest network combined – which in practice requires controlling more than 50% of total network hash power, a so-called 51% attack. Consider Alice’s 2 BTC payment to Bob recorded in Block 100. An attacker wanting to rewrite it to 20 BTC must recompute Block 100’s Proof of Work, then Blocks 101 through 110 (and beyond) as well, while honest miners are simultaneously extending the real chain further ahead – on a large network such as Bitcoin, this is practically impossible.

Significant energy cost. Proof of Work requires miners to perform millions, billions, or trillions of hash calculations, which demands specialized hardware (ASICs, or Application-Specific Integrated Circuits, purpose-built to compute SHA-256 as efficiently as possible), continuous electricity, and cooling. A single ASIC miner can compute roughly 100 trillion hashes per second, compared to about 1 billion per second for a GPU and about 50 million per second for a CPU which is exactly why Bitcoin mining today is dominated by ASIC hardware rather than general-purpose computers. If 100,000 ASIC miners each draw 3 kW continuously, that is 300 megawatts of continuous electricity consumption comparable to a small country's electricity use once summed across a network the size of Bitcoin's. This trade-off extremely strong, battle-tested security purchased at a very high energy cost is precisely what motivated the search for more energy-efficient alternatives such as Proof of Stake, which cuts energy consumption by more than 99% compared to Proof of Work.

4.6 The 51% Attack

A 51% attack occurs when a single miner, or a coordinated group of miners, controls more than half of the network's total hash power. With that majority, an attacker can produce blocks faster than the rest of the network and potentially reverse their own recent transactions or double-spend coins. Crucially, even a successful attacker cannot create new coins out of nothing, steal coins belonging to other users, or change the protocol's rules the attack is limited to manipulating the ordering and finality of the attacker's own recent transactions.

This attack is hard, not impossible. Suppose honest miners control 55% of the network's hash power and an attacker controls 45%; the attacker cannot out-produce the honest network and the attack simply fails. Succeeding requires acquiring genuinely more hash power than the rest of the network combined, which on a large chain means purchasing or controlling a massive quantity of mining hardware and electricity.

It is also self-limiting. A successful attack tends to damage confidence in the cryptocurrency itself, which can cause its price to fall meaning the attacker's own remaining coin holdings (mined honestly or otherwise) lose value at the same time. An attacker who owns 100 BTC and successfully double-spends 10 BTC, but in doing so triggers a 40% price crash, may end up losing more value on their remaining 100 BTC than they gained from the attack itself.

Finally, smaller chains are more at risk. A network with a much lower total hash rate is far cheaper to attack than Bitcoin. If SmallCoin's total network hash rate is 100 TH/s and Bitcoin's is roughly 700 EH/s, an attacker needs to acquire only slightly more than 50 TH/s to attack SmallCoin, compared to over 350 EH/s to attack Bitcoin – a difference of many orders of magnitude in cost, which is why smaller blockchains are meaningfully more vulnerable to this kind of attack.

Worked Example: A Simplified Proof of Work Puzzle

Target: the block hash must begin with three zeros, e.g., “000...”. The miner hashes the block header with nonce = 0, getting “7F3A...” – no match. The miner tries nonce = 1, 2, 3, and so on, potentially repeating this billions of times. Eventually, nonce = 1,847,213 produces “000C9B...” the target is met. Other nodes then verify this with a single hash computation, essentially instant compared to the miner's long search.

Part 2: Beyond Proof of Work – Replacing Computation with Economic Commitment

4.7 The Energy Question: Proof of Work vs. Proof of Stake

Under Proof of Work, security comes from real-world computation and electricity – verifiably costly, but at the cost of power consumption comparable to some mid-sized countries. Under Proof of Stake, validators lock up (stake) coins instead of burning electricity, and misbehaving validators simply lose part of their stake; this approach cuts energy use by more than 99%. Bitcoin and Litecoin use Proof of Work; Ethereum (since 2022) and Cardano use Proof of Stake.

4.8 How Proof of Stake Works

- 1. Stake collateral.** Validators lock up a quantity of the network's native cryptocurrency as collateral, demonstrating economic commitment to the network.
- 2. Get selected.** The protocol pseudo-randomly selects a validator to propose the next block, with the odds of being chosen typically weighted by the size of a validator's stake.
- 3. Earn or lose.** Honest validators earn staking rewards; dishonest validators risk losing part or all of their staked funds through slashing.

There is no puzzle to solve in Proof of Stake the economic stake itself, combined with the credible threat of slashing, provides the security guarantee. Suppose Alice stakes 100 ETH and Bob stakes 50 ETH. If Alice is selected to validate the next block, she verifies transactions, creates the block, and earns staking rewards; if she instead cheats, part of her 100 ETH can be slashed.

4.9 Slashing

Slashing is the penalty in Proof of Stake for dishonest or protocol-violating behavior: the network can forfeit part or all of a validator's staked funds. This can happen if a validator proposes an invalid block, or "double-signs" signs two conflicting blocks at the same block height, which threatens the integrity of the chain since only one block can legitimately occupy a given height. Suppose Validator A has staked 32 ETH and, at block height 500, signs both Block 500A and Block 500B, which contain different, conflicting transactions. The network detects this dishonest behavior and may slash part or in severe cases, all of the validator's 32 ETH stake, and may remove them from further participation. Because validators have real, valuable collateral at risk, slashing creates a strong financial incentive to behave honestly, replacing wasted computation with real financial risk as the network's security mechanism.

4.10 Ethereum's "The Merge" (2022)

In September 2022, Ethereum completed a transition from Proof of Work to Proof of Stake the most prominent real-world migration of its kind. The change reduced Ethereum's energy consumption by more than 99% while enabling faster block finality. Its significance goes beyond Ethereum itself: it proved that a major, live network handling enormous economic value could swap out its core security model without stopping operation, which removed what had been the strongest argument that blockchains must, of necessity, be energy-intensive.

Part 3: Other Consensus Models Different Trade-offs for Different Needs

4.11 Delegated Proof of Stake (DPoS)

Rather than everyone validating blocks, token holders vote to elect a small number of trusted delegates who validate transactions and produce blocks on the community's behalf. If a delegate performs poorly or acts dishonestly, token holders can vote them out and elect a replacement.

Suppose a blockchain has 100,000 token holders who, instead of all validating transactions themselves, vote for 21 delegates; those 21 delegates then validate all transactions and produce new blocks. This achieves substantially higher throughput than open mining or open staking, because fewer validators means blocks can be created faster but it concentrates power in a small elected group, which is a greater centralization risk than open participation. EOS is a well-known example of a blockchain using DPoS.

4.12 Practical Byzantine Fault Tolerance (PBFT)

PBFT allows a network to reach consensus even if some participants behave maliciously, and it is commonly used in private or permissioned blockchains among a known, trusted set of validators. A validator proposes a new block; all validators exchange multiple rounds of messages to verify it; if more than two-thirds of the validators agree, the block is immediately and finally added to the blockchain with no need to wait for additional, probabilistic confirmations the way Proof of Work requires. Suppose ten banks form a permissioned blockchain, each running a PBFT-compatible validator node. If seven or more of the ten (more than two-thirds) approve a proposed block, it is immediately accepted and cannot subsequently be reversed. PBFT tolerates fewer than one-third of validators behaving maliciously or failing with 10 validators, it can continue operating correctly even if up to three behave badly. The main disadvantage is scalability: as the number of validators grows, the volume of message exchange between them grows rapidly, making PBFT poorly suited to very large, open validator sets. Hyperledger Fabric uses PBFT-like consensus mechanisms in permissioned deployments.

4.13 Proof of Authority (PoA)

In Proof of Authority, a fixed set of pre-approved, identity-verified validators produce blocks instead of mining or staking, validators are selected because they are known and authorized. A company might create a private blockchain in which only five approved administrators are allowed to validate transactions and create new blocks. Because validator identities are publicly known, reputational risk substitutes for the financial or computational cost that secures other mechanisms misbehaving validators risk their real-world reputation rather than staked funds or wasted computation. This makes PoA well suited to private or consortium chains where participants already know and are accountable to one another.

A note on what validators actually are in practice: they are not people manually approving each transaction one by one. A validator is a computer running blockchain-specific software (for example, a Hyperledger Fabric peer or orderer, or an Ethereum client such as Geth), holding a private key used to sign blocks and messages, and connected to the network to exchange consensus messages with other validators automatically. The organization that owns a given validator – a bank, a company, a university – manages that server, but the actual moment-to-moment validation work is automated.

4.14 The Pattern: Speed vs. Centralization

The smaller and more well-known the validator set, the faster the network tends to be – but the more centralized it also becomes. Proof of Work and open Proof of Stake keep validation permissionless and widely distributed, at the cost of speed. DPoS, PBFT, and PoA all narrow the validator set in order to gain speed and immediate finality, at the cost of decentralization.

Part 4: Comparing the Models

4.15 Side-by-Side Comparison

	Proof of Work	Proof of Stake	Delegated / Authority
Security Basis	Computational cost	Staked capital + slashing	Known, trusted validators
Energy Use	Very high	Very low	Very low
Speed / Finality	Slower, probabilistic	Faster	Fastest
Decentralization	High	High (stake-dependent)	Lower

4.16 The Blockchain Trilemma

The blockchain trilemma states that it is difficult for any blockchain to fully maximize all three of the following properties simultaneously: decentralization (many independent participants share control), security (the network resists attacks and fraud), and scalability (the network can process a high volume of transactions quickly). Improving one property typically comes at the expense of one or both of the others.

Blockchain	Main Strength	Trade-off
Bitcoin (Proof of Work)	Very high security and decentralization	Lower transaction throughput (about 7 transactions/second) and high energy use
Ethereum (Proof of Stake)	Lower energy consumption and improved scalability	Security depends on staking economics and validator incentives
Private blockchain (PBFT/PoA)	Very fast transaction processing, high security among known parties	Less decentralized, since only approved participants validate blocks

Worked Example: A Miner Finds Block 105

1. The miner gathers pending transactions and builds a candidate block header.
2. Starting at nonce = 0, the miner hashes the header with SHA-256.
3. The output “9F2A...” does not meet the target, so the nonce increments and the header is hashed again and again.
4. After roughly four trillion attempts, nonce = 3,918,204,552 finally produces a hash with enough leading zeros to meet the target.
5. The miner broadcasts the block; every other node hashes it once to confirm the result, then accepts it into their own copy of the chain almost instantly.

Chapter 4 Recap: Key Takeaways

Mining and Proof of Work. Mining is a race to find a nonce whose resulting block hash meets the network’s difficulty target. The puzzle is deliberately expensive to solve but instant to verify, and that asymmetry is the basis of the network’s security. Miners are paid through the block reward and transaction fees, which aligns their financial incentives with honest behavior. A 51% attack is possible in theory but costly in practice on a large network, and the very high energy cost of Proof of Work is what motivated the search for alternatives.

Consensus mechanisms more broadly. Consensus mechanisms solve a core problem: how does a decentralized, mutually distrusting network agree on one shared version of the truth? Proof of Stake replaces computation with staked capital and slashing penalties, and is far more energy-

efficient it is Ethereum's current model. Delegated Proof of Stake, PBFT, and Proof of Authority all trade some decentralization for speed, and are each suited to different network needs. The blockchain trilemma decentralization, security, and scalability means every mechanism involves trade-offs; there is no single "best" choice independent of context.

Discussion Questions and Model Answers

1. Why is it computationally easy to verify a mined block but hard to find one in the first place?

Finding a valid hash requires brute-force trial and error: because SHA-256 exhibits the avalanche effect, there is no shortcut that predicts which nonce will produce a hash below the difficulty target, so a miner must simply try nonce after nonce potentially billions or trillions of times until one succeeds. Verifying a proposed solution, by contrast, requires only a single hash computation: another node hashes the same block header using the winning nonce and checks whether the result falls below the target. This asymmetry between an expensive search and a cheap check is precisely what makes Proof of Work secure.

2. If Bitcoin's block reward keeps halving toward zero, what will eventually motivate miners to keep mining?

As the block reward approaches zero through successive halvings, transaction fees are expected to make up an increasingly large share of miner revenue. Users will continue to pay fees to have their transactions included promptly, and as block-reward income shrinks, fee income is designed to become the dominant, and eventually the primary, incentive keeping miners securing the network.

3. Compare the economic incentives of Proof of Work and Proof of Stake what is one strength and one weakness of each?

Proof of Work's strength is that its security is grounded in real, externally verifiable, physical cost electricity and hardware which has proven battle-tested over many years; its weakness is the very large energy consumption this requires. Proof of Stake's strength is that it achieves strong security at a small fraction of the energy cost, since validators risk their own staked capital rather than burning electricity; its weakness is that its security is more directly tied to the economics of the staked asset itself for example, questions about validator concentration among large stakeholders,

or about the incentives created when a large share of a token's total supply is staked by a small number of participants.

4. Using the blockchain trilemma, explain why no single consensus mechanism can maximize decentralization, security, and scalability at once.

The trilemma holds that decentralization, security, and scalability pull against one another as design goals: widening the validator set (favoring decentralization) tends to slow down agreement (hurting scalability), while narrowing the validator set to gain speed (favoring scalability) concentrates control among fewer participants (hurting decentralization). Bitcoin, for instance, maximizes decentralization and security but processes only around 7 transactions per second; a permissioned PBFT network maximizes speed and security among its known members but sacrifices open decentralization. Because strengthening any one property tends to place pressure on at least one of the other two, no consensus mechanism has yet fully maximized all three simultaneously every real design represents a considered trade-off rather than a universal solution.

Chapter 5: Types of Blockchain Networks

Public · Private · Consortium · Hybrid

Learning Objectives

By the end of this chapter, you should be able to:

- Distinguish public, private, consortium, and hybrid blockchain networks in terms of who can join, who controls them, transaction speed, and transparency.
- Explain the difference between a blockchain network type (an architectural and governance choice) and a blockchain platform (the software used to build that network).
- Weigh the advantages and disadvantages of each network type.
- Apply a practical decision framework to choose the appropriate network type for a given business scenario.

Blockchain Platforms vs. Blockchain Network Types

Before comparing the four network types, it is worth clearing up a common point of confusion: public, private, consortium, and hybrid are architectural and governance models, not four blockchain systems that already exist somewhere waiting to be joined. An organization can either build its own blockchain network from scratch, using a platform such as Hyperledger Fabric, Ethereum, Corda, or Quorum, or it can join a network that another organization or community has already established. Bitcoin and Ethereum are examples of existing public networks that anyone can join without building anything new. A bank, in contrast, might build its own private blockchain on Hyperledger Fabric to manage internal financial records; several banks might collaborate to build a consortium blockchain that they jointly govern; and a government agency might build a hybrid blockchain that combines private and public components.

Hyperledger Fabric, Ethereum, Corda, and Quorum are blockchain platforms—the software, tools, and infrastructure used to build and operate a network—not network types in themselves. Public, private, consortium, and hybrid describe how a network is organized and governed; the platforms are the technologies used to build networks of any of those types.

Blockchain Platform	Typical Network Type	Developed / Maintained By	Common Use Cases
Ethereum	Public (can also be run privately)	Ethereum Foundation	Cryptocurrencies, smart contracts, DeFi, NFTs
Hyperledger Fabric	Private, Consortium	Linux Foundation	Supply chain, healthcare, banking, enterprise applications
Corda	Private, Consortium	R3	Banking, insurance, financial services
Quorum	Private, Consortium	Originally J.P. Morgan, now ConsenSys	Enterprise finance, digital assets, confidential business transactions

A simple analogy: public, private, consortium, and hybrid are the type of house you want to build (an apartment block, a private home, an office building, a mixed-use building); Hyperledger Fabric, Ethereum, Corda, and Quorum are the construction tools and materials bricks, steel, concrete, machinery used to build it. An organization first decides which network type best fits its needs, and only then chooses which platform to build it with.

Type 1 Public Blockchain: Open to Anyone, Trusted by Design

A public blockchain is a fully open, permissionless network: anyone can read the ledger, submit a transaction, or participate in validation, with no approval required. Bitcoin and Ethereum are the two canonical examples. Security and trust in a public blockchain come from decentralization itself thousands of independent nodes worldwide, with no single point of control.

To see why this matter, contrast it with a traditional centralized system. In a bank, a single central authority stores all account records, approves every transaction, and can modify records at will; if the bank's server fails or is compromised, the entire system is affected, because the bank is a single point of control. Now imagine, instead of one bank, more than 20,000 independent nodes spread across the world in the United States, Japan, Germany, Brazil, South Africa, and elsewhere each holding an identical copy of the blockchain and independently verifying every transaction. No single node controls the network; this is decentralization in practice.

Trust follows the same logic. When Alice sends 2 BTC to Bob, the transaction is not sent to a single bank for approval; it is sent to thousands of independent nodes, each of which checks independently whether Alice's signature is valid, whether she actually owns the funds, and whether she has already spent them. If a dishonest node falsely claims that Alice sent 100 BTC instead of 1 BTC, the other 19,999 honest nodes simply reject that false claim, because it violates the network's rules – a single dishonest node cannot force the rest of the network to accept its version of events. Trust, in other words, comes from the consensus of thousands of independent computers, not from trusting any one organization.

This same structure also removes any single point of failure. If one bank shuts down, is hacked, or loses power, everyone who depends on it is affected. If, instead, one node among 20,000 goes offline, the remaining 19,999 continue operating exactly as before, and transactions continue normally. A helpful classroom analogy: in a class of 100 students, a centralized attendance system has only the class representative keeping the attendance notebook – if that notebook is lost, the record is gone. In a decentralized version, every student keeps an identical copy of the attendance list; if one student's copy is lost, the other 99 copies still exist, and it becomes almost impossible for any single student to secretly falsify the record without the inconsistency being noticed by everyone else.

Type 2 Private Blockchain: One Organization, Full Control

A private blockchain is controlled by a single entity, which decides who may join, what data participants can see, and how validation works. A private, internal enterprise ledger used by a single bank or retailer to manage its own records is a typical example. A private blockchain trades away decentralization in exchange for speed, privacy, and organizational control – in spirit, it sits much closer to a traditional centralized database than to Bitcoin. Because only a limited number of trusted, authorized participants validate transactions, transactions can be processed much faster than on a public network, and sensitive information stays accessible only to those the organization authorizes. A bank, for example, might use a private blockchain to manage customer transactions, with only the bank's own authorized servers validating them – fast, secure, and fully under the bank's control.

It is worth noting explicitly why a private blockchain still counts as a “blockchain” at all, despite this centralization: it still groups transactions into blocks, links each block to the previous one via cryptographic hashes, protects transactions with digital signatures, and requires a consensus mechanism to approve new blocks, maintaining a shared, immutable, auditable ledger across its authorized nodes. What differs from a public blockchain is only who controls and participates in the network – not the underlying technology stack.

Type 3 Consortium (Federated) Blockchain: Shared Control Among Known Partners

A consortium blockchain shares control among a pre-selected group of organizations – for example, a group of banks or hospitals – rather than one company or the general public. Typical deployments run on Hyperledger Fabric: a multi-bank trade finance network, or a hospital data-sharing consortium, for instance. Each member organization typically operates one or more nodes, and the consensus rules are agreed upon collectively by the whole consortium. An interbank payment network is a good illustration: several banks jointly operate a blockchain to securely process transactions between themselves, share financial records, and verify payments – without depending on any single central authority, and without exposing the data publicly.

Type 4 Hybrid Blockchain: The Best of Both Worlds, Selectively

A hybrid blockchain combines elements of the public and private models: some data or transactions remain private and tightly controlled, while others are published to a public chain for outside verification. The guiding principle is “public where useful, private where needed.” A government land registry, for example, might keep detailed ownership records private while periodically publishing a cryptographic hash of those records to a public chain, so that outsiders can verify the records have not been tampered with – without ever revealing the confidential underlying data.

The mechanics work in three steps. First, sensitive records – patient names, medical histories, national identification numbers, financial details – are stored on a private blockchain accessible only to authorized users. Second, rather than publishing that confidential data directly, the organization generates a hash of it – a unique digital fingerprint that reveals nothing about the underlying content but changes completely if the content changes. Third, that hash alone is published to a public blockchain (Ethereum, for instance), along with a timestamp. Anyone can

then verify the integrity of a record by retrieving the original private record, generating its hash again, and comparing it against the hash stored publicly: if the two hashes match, the record has not been altered; if they differ, it has been tampered with.

This pattern applies broadly. A land registry can store ownership records privately while publishing only their hashes, letting anyone verify the records are unaltered without exposing confidential ownership details. A shipping company can keep shipment details private while publishing hashes so that customers and auditors can independently verify record integrity. In each case, the private blockchain provides confidentiality, restricted access, and fast processing, while the public blockchain provides broad accessibility, transparency, and tamper-proof verification combined, they let an organization protect sensitive information, ensure data integrity, prevent unauthorized modification, and still enable public verification, all at once.

A hybrid blockchain's security depends heavily on getting the split right: if an organization mistakenly publishes something that should have stayed private, confidentiality is broken; if it hides too much, transparency and trust suffer. In a hospital's hybrid system, for example, patient names and medical records should remain strictly private, while only a record identifier and a cryptographic hash are published allowing anyone to verify a record's authenticity without ever seeing the patient's confidential information.

Comparing the Four Network Types

	Public	Private	Consortium	Hybrid
Who Can Join	Anyone	One organization	Pre-selected group	Mixed access
Control	Fully decentralized	Fully centralized	Shared among members	Partially shared
Transaction Speed	Slower	Fastest	Fast	Variable
Transparency	Fully public	Fully private	Private to members	Selective

On control. Control refers to who has the authority to manage the network, validate transactions, and make governance decisions. A public blockchain like Bitcoin is fully decentralized: thousands

of nodes collectively validate transactions, and no single organization controls the process. A private blockchain, such as a hospital's internal patient record system, is fully centralized: the hospital owns and controls every aspect of who can join, read, write, and update the system. A consortium blockchain, such as five banks jointly approving software updates and validating transactions, distributes governance among the member organizations. A hybrid blockchain splits control: an organization retains full control over sensitive private data, while selected information is shared publicly for verification.

On transaction speed. Public networks are slower because thousands of nodes must participate in reaching consensus. Private networks are fastest because only a small number of trusted, internal nodes need to validate. Consortium networks are fast, since only the consortium's own members validate. Hybrid networks are variable, since private operations run quickly while publishing hashes to a public blockchain depends on that public network's own speed.

On transparency. Public networks are fully public anyone can view every transaction. Private networks are fully private only users the owning organization authorizes can access the data. Consortium networks are private to their members visible only to participating organizations. Hybrid networks offer selective transparency: sensitive information stays private, while chosen information, such as a hash, is made public.

Advantages and Disadvantages

Public vs. Private.

Public	Private
Maximum decentralization and censorship resistance	High throughput and low latency
Fully transparent and independently auditable	Full control over data privacy and access
Slower transaction throughput at scale	Single point of failure / control
No privacy all transaction data is visible	Limited independent verifiability by outsiders

“Slower transaction throughput at scale” deserves a concrete illustration. Bitcoin can process only around 7 transactions per second. If 100,000 people attempt to send Bitcoin transactions simultaneously during a busy event, most cannot be confirmed immediately; they queue in the mempool, and users may wait several minutes or considerably longer during heavy congestion before their transaction is finally included in a block. As transaction demand grows, a public blockchain simply cannot process everything instantly, and delays result.

Consortium vs. Hybrid.

Consortium	Hybrid
Balances decentralization with efficiency	Flexible confidential where needed, public where useful
Well suited to known, multi-party collaboration	Can prove integrity without revealing all data
Requires coordination and trust among members	More complex architecture to design and maintain
Less open than a public network	Security depends on getting the public/private split right

Real-World Applications

Public blockchains dominate cryptocurrencies, DeFi, NFT marketplaces, and public DAOs. A public DAO might, for example, let 10,000 members worldwide, each holding governance tokens, vote to release \$100,000 from a shared treasury to fund an environmental project with a smart contract automatically releasing the funds once the vote passes, rather than a CEO or board deciding unilaterally.

Private blockchains are most common for internal corporate record-keeping and single-company asset tracking. Consortium blockchains show up in multi-bank trade finance, hospital data-sharing, and supply chain consortia. Hybrid blockchains are increasingly used for government land records and cross-border logistics that require selective disclosure.

Choosing the Appropriate Network: A Practical Decision Framework

A short set of questions can guide the choice in practice: Do unrelated strangers need to transact without a trusted intermediary? → Public. Is the use case purely internal to one organization, with no need for outside trust? → Private. Does a known group of organizations need to collaborate without any single one of them controlling the whole system? → Consortium. Do you need most data to stay private, but occasional, verifiable proof of integrity visible to outsiders? → Hybrid.

Worked Example: Picking a Network for a Real Scenario

Five competing shipping companies want to jointly track container locations and customs status. They do not fully trust each other, since they are direct competitors, but they also do not want this operational data to be fully public to competitors or the general public. A private blockchain is ruled out immediately, because it would require one company to own and control the system, and the others — being competitors — are unlikely to accept that. A fully public blockchain is also unsuitable, because it would expose competitively sensitive shipment data to everyone, including non-participants and rivals outside the group entirely. The best fit is a consortium blockchain: each shipping company runs its own node, and the consensus rules governing the network are agreed upon collectively. This provides the right balance of security, privacy, decentralization, and collaboration — no single company dominates the network, confidential business data stays away from the public, and the five companies can still securely share the operational information they actually need to share.

Chapter 5 Recap: Key Takeaways

- Public blockchains maximize decentralization and transparency, at the cost of speed and privacy; Bitcoin and Ethereum are the best-known examples.
- Private blockchains maximize speed and control for a single organization, but sacrifice the decentralization that defines public networks.
- Consortium blockchains share control among a known group of organizations, often the best fit for multi-party enterprise collaboration.
- Hybrid blockchains selectively combine public and private elements, letting organizations keep most data confidential while still proving integrity to outsiders.

Discussion Questions and Model Answers

1. Why might a group of competing companies prefer a consortium blockchain over either a fully public or fully private one?

A consortium blockchain lets competing companies share control of the network while protecting confidential business information. Unlike a private blockchain, no single company owns or controls it, which builds mutual trust among participants who are otherwise rivals. Unlike a public blockchain, sensitive business data is not visible to the general public or to outside competitors, which preserves confidentiality. Five shipping companies jointly using a consortium blockchain to track shipments, each running a validator node, illustrates the pattern: shipment data is shared only among the participating companies and relevant customs authorities, not with competitors or the public.

2. Describe a real-world scenario where a hybrid blockchain's selective transparency would be more valuable than full public transparency.

A university storing student academic records provides a good example. The actual transcripts, grades, and personal information remain private, accessible only to authorized staff and the students themselves. A cryptographic hash of each transcript, however, is published on a public blockchain. Employers can then verify that a transcript is authentic by comparing its hash against the publicly stored one without ever viewing the student's confidential academic details.

3. If decentralization is one of blockchain's defining strengths, why do private blockchains still get used in practice?

Many organizations prioritize speed, privacy, and centralized control over the benefits of full decentralization, particularly when handling confidential information at high transaction volumes. A commercial bank managing customer accounts on a private blockchain, for instance, values fast processing, confidentiality of customer information, and regulatory compliance more highly than decentralization for its own sake and a private blockchain still preserves the underlying technical benefits of blockchain (linked blocks, cryptographic hashing, digital signatures, consensus-based validation, an immutable and auditable ledger), even though only the bank's own authorized servers participate in it.

Chapter 6: Blockchain Platforms and Ecosystems

Bitcoin · Ethereum · Hyperledger · Smart Contracts

Learning Objectives

By the end of this chapter, you should be able to:

- Explain why Bitcoin’s scripting language was deliberately kept simple, and what the Lightning Network adds on top of it.
- Explain the Ethereum Virtual Machine, gas, and the rollup-centric scaling roadmap.
- Explain what a permissioned framework like Hyperledger Fabric offers that public blockchains do not, and identify other notable enterprise platforms.
- Explain smart contracts using the vending machine analogy, and describe the path from Solidity source code to a deployed, immutable contract.
- Compare Bitcoin, Ethereum, and Hyperledger Fabric side by side and match each to an appropriate use case.

Bitcoin: The Pioneer Network, Digital Cash, Deliberately Simple

Bitcoin was designed with one primary purpose: to serve as digital cash, letting people send and receive money securely over the internet. It is deliberately built for one job. Bitcoin focuses on secure, reliable money transfers, not on running complex applications; when Alice sends 0.5 BTC to Bob, the network verifies and permanently records that transaction, and that is essentially the whole scope of what Bitcoin’s protocol is designed to do.

Bitcoin’s scripting language is deliberately non-Turing-complete meaning it can perform only specific, limited tasks and cannot execute unbounded or arbitrarily complex programs. It can express conditions such as “these coins can only be spent if a valid digital signature is provided,” but it cannot run loops or general-purpose logic the way a full programming language can. This is a considered trade-off: limiting what the scripting language can do also limits the ways it can go wrong, reducing the risk of software bugs and security vulnerabilities in a system that, by design, handles enormous amounts of value.

Bitcoin's supply is capped at 21 million coins, enforced by the same halving mechanism introduced in Chapter 4 the mining reward is cut in half roughly every four years ($50 \rightarrow 25 \rightarrow 12.5 \rightarrow 6.25 \rightarrow 3.125$ BTC and onward), slowing the creation of new coins over time. Because Bitcoin cannot be printed at will the way traditional currencies can, and because its supply is provably finite, many people view it as “digital gold”: a long-term store of value rather than an everyday transactional currency.

That framing raises an obvious question: if Bitcoin's base layer is deliberately conservative and slow, how does it support fast, everyday payments? The answer is the Lightning Network, a Layer 2 protocol that enables fast, low-cost Bitcoin payments through off-chain payment channels, while the base layer itself stays simple and conservative. Suppose Alice buys coffee from the same café every day. Without Lightning, every single payment would need to be recorded directly on the Bitcoin blockchain, with all the fees and confirmation delays that involves. With Lightning, payments between Alice and the café happen instantly through a payment channel opened between them, and only the final net balance is ever recorded on the main Bitcoin blockchain. In short, Bitcoin's core design prioritizes security, reliability, and scarcity over flexibility, while complex or high-frequency use cases are handled by Layer 2 technologies built on top, leaving the base chain itself secure and stable.

Ethereum: The Programmable Blockchain, “A Computer the Whole World Shares”

Unlike Bitcoin, Ethereum is designed to run applications smart contracts on the blockchain itself, which is why it is often described as a computer the whole world share.

The Ethereum Virtual Machine (EVM). The EVM is the software that executes smart contracts on every Ethereum node. Being Turing-complete means the EVM can, in principle, run almost any program, which makes Ethereum dramatically more flexible than Bitcoin. This single innovation catalyzed the emergence of DeFi, NFTs, DAOs, and the broader dApp ecosystem covered in later chapters. When a developer builds a decentralized voting application and users cast votes, the EVM executes the same smart contract logic on every Ethereum node, guaranteeing everyone arrives at the same result.

Gas and Ether (ETH). Every operation the EVM performs consumes gas a measure of the computational work required. ETH, Ethereum's native cryptocurrency, is the currency used to pay

for that gas. A useful mental model: gas is the fuel, and ETH is the money used to buy the fuel. Gas exists for two reasons. First, it compensates the validators who process transactions and execute the underlying computation. Second, it prevents abuse—for instance, a poorly written contract containing an infinite loop (`while (true) { // run forever }`) would, without gas, simply lock up the network’s resources forever; with gas, execution halts the moment a transaction runs out of gas, capping the damage any single transaction can do. The transaction fee formula is simple: **Transaction Fee = Gas Used × Gas Price**. If a transaction uses 21,000 gas at a price of 10 Gwei per unit, the fee is 210,000 Gwei, paid in ETH.

Ethereum’s scaling roadmap. Layer 1—the main Ethereum chain—is secure and decentralized, but has limited raw throughput. Layer 2 networks, particularly rollups, handle most transaction execution off the main chain and then settle their results back to Layer 1, while still inheriting Layer 1’s underlying security guarantees. This “rollup-centric roadmap” shifts the bulk of execution off the main chain, dramatically increasing overall throughput without sacrificing the security the base layer provides. (Chapter 8’s discussion of Layer 2 rollups and their new trust assumptions builds directly on this idea.)

Hyperledger and Enterprise Blockchain: When Permissionless Isn’t the Goal

Hyperledger Fabric is an open-source, permissioned framework maintained by the Linux Foundation, in which participants must be explicitly authorized to join—built specifically for enterprise and consortium use cases where full network-wide transparency is a liability rather than a feature. Its most distinctive feature is channels, which let subsets of participants conduct private, confidential transactions invisible to everyone else on the network—extremely valuable in domains like supply chain management and healthcare, where different parties legitimately need to see different subsets of the same underlying data. Fabric also supports a modular, configurable consensus mechanism, rather than mandating one fixed approach the way Bitcoin mandates Proof of Work.

Other notable enterprise platforms round out the picture.

Hyperledger Besu is an Ethereum-compatible client suited to both public and private deployments, bridging the divide between fully public and fully permissioned networks.

Corda, built by R3, is designed primarily for financial services and uses a notary-based consensus model tailored to regulated transactions. The pattern across all of these enterprise platforms is consistent: they trade away some decentralization in exchange for privacy, throughput, and regulatory fit—an appropriate trade when the participants already know and trust one another, which directly echoes the centralization-versus-decentralization theme first raised in Chapter 1.

Smart Contracts: Concept and Architecture

Nick Szabo coined the term “smart contract” in the 1990s, well before blockchain technology existed to actually implement the idea. The most memorable way to understand a smart contract is the vending machine analogy: a vending machine automatically enforces an agreement—insert the correct payment, and the machine releases the product, with no cashier needed to mediate the exchange. A smart contract works the same way: code automatically enforces an agreement once its conditions are met, with no trusted intermediary required. Other everyday examples share the same structure—a parking meter, or an automated ticket dispenser.

From code to deployed contract follows three steps. First, developers write the contract’s logic in a high-level language such as Solidity. Second, that code compiles down into EVM bytecode, which the network can actually execute. Third, once the contract is deployed to a blockchain address, its code is typically immutable—it runs exactly as written, forever. This immutability is simultaneously a strength and a risk: it guarantees the contract’s behavior can never be secretly altered after deployment, but it also means that any bug present at the moment of deployment becomes permanent, unless the developers deliberately designed an upgrade path into the contract in advance. (Chapter 8 covers what happens when this goes wrong, using the 2016 DAO hack as its central case study.)

It is worth flagging, even briefly, that smart contracts can do more than just execute logic in isolation: they can also hold funds directly, call other contracts, and emit events that off-chain applications can monitor—capabilities that Chapter 7 explores in much greater depth when it turns to hands-on dApp development.

Comparing the Platforms

	Bitcoin	Ethereum	Hyperledger Fabric
Network Type	Public, permissionless	Public, permissionless	Permissioned
Smart Contracts	Limited scripting	Full (Turing-complete)	Full (“chaincode”)
Primary Use Case	Digital cash / store of value	DeFi, NFTs, general dApps	Enterprise / consortium
Consensus	Proof of Work	Proof of Stake	Configurable

Putting It into Practice: Which Platform, and Why?

If you need maximum security and monetary predictability, Bitcoin’s conservative, deliberately limited scripting and hard-capped supply make it well suited to value storage and simple payments. If you need programmability for complex applications, Ethereum’s Turing-complete EVM supports DeFi, NFTs, DAOs, and essentially any custom application logic you can encode. If you need privacy among a known group of business partners, Hyperledger Fabric’s permissioned model and channel architecture let you share data selectively only with the parties who need to see it rather than broadcasting it network-wide. A quick worked scenario: a hospital consortium wanting to share patient records among a known group of member hospitals, without exposing that data publicly, is a strong fit for Hyperledger Fabric precisely because of its channel architecture and permissioned access model.

Chapter 6 Recap: Key Takeaways

- Bitcoin prioritizes security and monetary soundness through deliberately limited, non-Turing-complete scripting, with the Lightning Network extending it toward fast, everyday payments off-chain.
- Ethereum’s Turing-complete EVM, paid for through gas, enabled smart contracts and the broader dApp ecosystem; its rollup-centric roadmap aims to scale throughput while preserving Layer 1 security.
- Hyperledger Fabric and similar permissioned frameworks trade openness for privacy, control, and regulatory fit, using mechanisms such as channels to share data selectively.

- Smart contracts, like a vending machine, automatically enforce an agreement once its conditions are met, with no intermediary required but their immutability once deployed is both their greatest strength and their sharpest risk.

Discussion Questions and Model Answers

1. Why was Bitcoin's scripting language deliberately designed to be non-Turing-complete, and what trade-offs does this involve?

Bitcoin's scripting language was kept deliberately limited to prioritize security and predictability over flexibility. A simple, non-Turing-complete language can express conditions such as signature verification without allowing arbitrary, unbounded programs to run, which sharply reduces the surface area for bugs and exploitable vulnerabilities in a system securing enormous amounts of value. The trade-off is flexibility: Bitcoin's scripting cannot support the kind of complex decentralized applications DeFi protocols, NFT marketplaces, DAOs that a Turing-complete platform like Ethereum can support natively.

2. Explain how gas functions on Ethereum, and why a Turing-complete blockchain needs a mechanism like this.

Gas measures the computational work required by each operation the EVM performs, and users pay for that gas in ETH, according to the formula $\text{Transaction Fee} = \text{Gas Used} \times \text{Gas Price}$. A Turing-complete blockchain needs a mechanism like gas because, without it, a poorly written or malicious contract containing an infinite loop or another runaway computation could consume network resources indefinitely, effectively halting the network for everyone. Gas caps the computational cost any single transaction can impose execution simply stops once a transaction's gas runs out while also compensating the validators who actually perform the computation.

3. What distinguishes a permissioned blockchain like Hyperledger Fabric from a permissionless network like Ethereum?

On a permissionless network like Ethereum, anyone can join, read the ledger, submit transactions, or participate in validation, with no approval required from any central party. On a permissioned network like Hyperledger Fabric, participants must be explicitly authorized before they can join the network at all, and the network additionally supports channels that allow subsets of authorized

participants to transact privately, invisible to other members of the same network. This makes Hyperledger Fabric far better suited to enterprise and consortium settings, where full network-wide transparency would actually be a liability rather than a benefit.

Chapter 7: Development of Decentralized Applications (dApps)

Solidity · Remix & Truffle · Web3.js · The dApp Lifecycle

Learning Objectives

By the end of this chapter, you should be able to:

- Describe the three-layer architecture of a typical dApp, and honestly assess how “decentralized” most real dApps actually are.
- Read and explain a simple Solidity smart contract, including state variables, functions, and the `view` keyword.
- Explain what payable functions, modifiers, and events are for.
- Compare Remix, Truffle, and Hardhat as development environments, and know when each is appropriate.
- Explain how a frontend, an ABI, Web3.js/Ethers.js, and a wallet provider such as MetaMask work together to connect a user interface to a deployed smart contract.
- Walk through the full dApp development lifecycle, from design through mainnet deployment and maintenance.

What Makes an App “Decentralized”? The Three-Layer dApp Architecture

A typical dApp is built from three layers, and it is useful to think of them as the path a single user action takes through the system. The frontend layer — HTML, CSS, and JavaScript frameworks such as React — is the user interface the person actually sees and clicks on. The connecting layer — libraries such as Web3.js or Ethers.js — lets that frontend read data from the blockchain and submit transactions to it. The smart contract layer, deployed on-chain, encodes the application’s actual core business logic and state. When a user clicks something in the frontend, the connecting layer translates that click into a blockchain transaction, and the smart contract layer executes it and stores the result permanently.

A healthy dose of realism is warranted here: very few dApps are entirely decentralized in practice. Frontend hosting is often still provided by ordinary, centralized web hosting services, exactly like any other website. IPFS exists as an emerging decentralized hosting alternative, but it remains far from universal. Many dApps also rely on centralized node providers to communicate with the

blockchain, rather than running their own full node. In practice, it is usually the smart contract layer being genuinely on-chain immutable, transparent, and executed by the network rather than by any single company's servers that makes an application meaningfully “decentralized” today, even when its frontend and infrastructure are not.

Introduction to Solidity: Your First Smart Contract

Consider a minimal example, a contract that simply stores a single number:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract SimpleStorage {
    uint256 storedData;

    function set(uint256 x) public {
        storedData = x;
    }

    function get() public view returns (uint256) {
        return storedData;
    }
}
```

`storedData` is a state variable it persists permanently on the blockchain, not merely in a program's temporary memory. `set(x)` writes to the blockchain, which costs gas, because it changes the contract's stored state. `get()` is marked `view`, meaning it is read-only and free to call it does not modify any state, so it does not consume gas when called externally. This `view` keyword distinction is one of the most common points of confusion for students new to Solidity, and it connects directly to the gas mechanics introduced in Chapter 6: any function that changes state costs gas, while any function that only reads state does not.

Solidity also offers a handful of features built specifically for the blockchain context. A function marked `payable` is allowed to receive Ether along with the call without this keyword, a function cannot accept incoming funds at all. Modifiers are reusable preconditions commonly used for access control that can be attached to multiple functions without duplicating the same check in each one. A classic example is an `onlyOwner` modifier, which checks that `msg.sender` matches the contract's designated owner before allowing a function to execute; without such a check, a `withdraw` function, for instance, could be called by absolutely anyone, not just the contract's

intended owner a serious security flaw of exactly the kind discussed in Chapter 8. Events let a contract log information that off-chain applications can efficiently monitor, without those applications having to constantly poll the blockchain’s full state.

Development Tools

Remix is a browser-based IDE requiring no installation at all: you can write, compile, debug, test, and deploy Solidity contracts directly from your web browser, making it an ideal entry point for anyone new to smart contract development. It is best suited for learning, prototyping, and quickly testing a single contract, and it includes built-in static analysis tools that flag common security vulnerabilities before deployment a feature that connects directly to the security discussion in Chapter 8.

Truffle and Hardhat represent the next step up, once a project outgrows a single contract and needs structured, repeatable testing and deployment across multiple networks. Truffle is a comprehensive framework offering a standardized project structure, a built-in testing framework, and “migrations” that manage deployment across different networks. Hardhat is valued particularly for powerful debugging detailed stack traces and console logging directly from Solidity code along with a flexible plugin ecosystem. Both require a local Node.js development environment, a meaningful step up in complexity from Remix, but both are built for production-scale projects rather than quick prototypes.

Connecting Frontends with Web3.js: Bridging the Browser and the Blockchain

Three pieces work together to let a frontend actually talk to a deployed smart contract. The ABI (Application Binary Interface) is a JSON description of the contract’s functions; it tells the frontend exactly how to communicate with the contract which functions exist, what parameters they take, and what they return. Web3.js or Ethers.js, JavaScript libraries, use that ABI to query blockchain data and to send transactions. A wallet provider, such as MetaMask, injects a provider into the web page; transactions are signed inside the wallet itself, so a user’s private key never leaves the wallet or touches the website’s own code.

This design directly preserves the security model introduced in Chapter 2: a user’s signature authorizes a given transaction, but their private key never touches the website’s code at all the

same digital-signature guarantees (authentication, integrity, non-repudiation) are simply being applied at the application layer, mediated by the wallet rather than by the raw blockchain protocol.

The dApp Development Lifecycle: From Idea to Mainnet

- 1. Design.** Decide what belongs on-chain and what belongs off-chain not every piece of application logic needs to live inside a smart contract, and keeping some logic off-chain can reduce gas costs and complexity.
- 2. Develop & test locally.** Iterate rapidly on a local test blockchain, where transactions are free and instant, allowing for quick experimentation.
- 3. Testnet deployment.** Test the application under conditions closely resembling production, but on a network where no real funds are at risk.
- 4. Security audit.** Combine manual review with automated analysis before deploying to mainnet this step is covered in depth in Chapter 8.
- 5. Mainnet & maintenance.** Deploy to the live network, followed by ongoing monitoring and carefully governed upgrades where an upgrade path was designed in from the start.

This lifecycle is iterative rather than strictly linear in practice: bugs discovered at the audit stage often send a development team back to the develop-and-test stage before another attempt at deployment.

Worked Example: A Simple Decentralized Voting dApp

1. The contract stores a mapping of proposal IDs to vote counts, and separately tracks which addresses have already voted, to prevent double voting.
2. A `vote(proposalId)` function checks that the caller has not already voted, increments the relevant vote count, and emits a `VoteCast` event.
3. The frontend, built with Web3.js, reads live vote counts directly from the contract's on-chain state.
4. When a user votes, MetaMask prompts them to review and sign the transaction before it is submitted.

5. The frontend listens for the VoteCast event and updates the displayed results in real time as new votes come in.

This example draws together nearly every concept in the chapter at once: contract state and functions, events, the ABI and Web3.js, MetaMask's role in signing, and the design decisions embedded in the lifecycle described above.

Chapter 7 Recap: Key Takeaways

- A dApp typically has three layers – a smart contract layer on-chain, a frontend layer, and a connecting layer (Web3.js/Ethers.js) though true decentralization varies considerably in practice, since frontend hosting and node access are often still centralized.
- Solidity contracts define state variables and functions; view functions read data for free, while state-changing functions cost gas.
- Remix is ideal for learning and prototyping; Truffle and Hardhat support larger, production-scale projects requiring structured testing and multi-network deployment.
- The dApp lifecycle runs from design, through local testing, testnet deployment, security audit, and finally mainnet deployment and maintenance – with bugs found late in the process often sending the team back to an earlier stage.

Discussion Questions and Model Answers

1. Why might a developer choose to keep certain application logic off-chain rather than encoding it entirely within a smart contract?

Every operation performed on-chain costs gas and is permanently, publicly recorded, so developers often keep logic off-chain when it does not strictly need blockchain guarantees – for instance, complex calculations that do not need to be trustlessly verifiable, user-interface state, or data that changes too frequently to sensibly store on-chain. Keeping such logic off-chain reduces gas costs, improves performance, and avoids permanently and publicly recording information that has no need to live on an immutable ledger, while reserving the smart contract itself for the specific logic – ownership, fund transfers, voting outcomes – that genuinely benefits from blockchain's trustless, tamper-evident guarantees.

2. Compare Remix and Truffle as development environments when would each be the more appropriate choice?

Remix is the more appropriate choice when learning Solidity, prototyping a single contract quickly, or testing an idea without any local setup, since it runs entirely in the browser with no installation required. Truffle becomes more appropriate once a project grows beyond a single contract and needs a standardized project structure, a proper automated testing framework, and repeatable “migrations” for deploying consistently across multiple networks (local, testnet, and mainnet) the kind of structured, production-scale workflow that a browser-based tool like Remix is not designed to support.

3. Describe the role of an ABI in enabling a frontend application to interact with a deployed smart contract.

The ABI (Application Binary Interface) is a JSON description of a smart contract’s functions their names, parameters, and return types. It tells a frontend application exactly how to construct valid calls to the contract, which is essential because the blockchain itself only understands raw bytecode. Libraries such as Web3.js or Ethers.js use the ABI to translate ordinary JavaScript function calls made by the frontend into the correctly formatted transactions or read calls the deployed contract expects, effectively acting as the technical bridge between a human-friendly user interface and the contract’s on-chain bytecode.

Chapter 8: Blockchain Security and Privacy

Vulnerabilities · Audits & Best Practices · Privacy Techniques

Learning Objectives

By the end of this chapter, you should be able to:

- Explain the 2016 DAO hack and the tension it revealed between strict immutability and the practical need to respond to security incidents.
- Identify and explain the most common classes of smart contract vulnerability: reentrancy, integer overflow/underflow, access control flaws, front-running, oracle manipulation, and gas-based denial of service.
- Explain the checks-effects-interactions pattern and why it defends against reentrancy.
- Describe what security audits, audited libraries, fuzz testing, and bug bounties each contribute, and why “audited” does not mean “safe.”
- Explain zero-knowledge proofs and related privacy-preserving techniques, and the tension they navigate between transparency and privacy.
- Explain why scaling solutions can introduce new categories of security risk.

Immutability and Data Integrity, Revisited: The 2016 DAO Hack

Chapter 1 introduced immutability as one of blockchain’s four defining pillars—a strength, but also a limitation, because genuine mistakes cannot be reversed. This chapter opens with the case that made that tension impossible to ignore. In 2016, an attacker exploited a reentrancy vulnerability (explained fully below) to drain a significant portion of the pooled funds held by “The DAO,” a large, high-profile decentralized investment fund built on Ethereum. The Ethereum community faced a genuine dilemma: accept the loss, preserving strict immutability as an inviolable principle, or intervene to reverse the theft, compromising that principle in order to prevent serious harm to a large number of ordinary participants.

The resolution split the community. A hard fork moved the stolen funds to a recovery contract, effectively rewriting recent history to undo the attack—this fork became what is known today simply as Ethereum. A minority of the community rejected this intervention on principle and continued running the original, unaltered chain, which continues to this day as Ethereum Classic.

There is no single, universally agreed “correct” answer to whether the hard fork was the right call; the value of the case study lies in understanding the trade-off itself, not in memorizing a verdict. What is not in dispute is that this event directly motivated the security best practices – audits, standardized patterns, audited libraries – discussed later in this chapter.

Common Smart Contract Vulnerabilities

Reentrancy attacks. A malicious contract repeatedly calls back into a vulnerable function before that function’s internal state updates have finished, allowing funds to be drained through repeated withdrawals within what is, from the blockchain’s perspective, a single transaction. This is the exact mechanism behind the DAO hack, and it is worth dwelling on in detail below.

Integer overflow / underflow. Arithmetic that exceeds a variable’s representable range can wrap around unexpectedly – for example, subtracting 1 from a value of 0 in an unsigned integer could, in vulnerable code, wrap around to an enormous positive number instead of producing an error. This class of bug is largely mitigated automatically in Solidity 0.8 and later, which reverts on overflow by default, but it remains a real risk in older, legacy contract code that predates this protection.

Access control flaws. Functions that should be restricted – for instance, to an administrator or contract owner – but lack proper permission checks, letting anyone call functions that were only ever meant to be called by an authorized party.

Front-running. Because pending transactions sit visibly in the public mempool before being confirmed, an attacker can observe a profitable pending transaction and submit their own transaction with a higher gas price, paying extra to have their transaction processed first and thereby profiting at the original transaction’s expense.

Oracle manipulation. Many contracts rely on external data feeds, called oracles, for decisions such as asset pricing. If an attacker can manipulate the underlying data source that an oracle reports, they can trigger contract behavior that is technically valid according to the contract’s code but is illegitimate given the manipulated input – for instance, tricking a lending protocol into believing collateral is worth more than it actually is.

Denial of service via gas limits. Unbounded loops, or external calls deliberately designed to fail, can prevent a contract from ever completing an operation it was intended to support, effectively locking up its functionality.

Because smart contracts frequently hold significant real value and are often immutable once deployed, they represent an especially high-stakes target for all of the vulnerability classes above.

Security Audits and Best Practices: Catching Bugs Before They're Permanent

Audits combine expert manual code review with automated static analysis tools that scan for known, previously catalogued vulnerability patterns. The single most important defensive pattern covered in this chapter is checks-effects-interactions: perform all necessary checks first, update the contract's internal state second, and only then make any external calls last. This ordering is the standard, well-established defense against reentrancy specifically.

The reasoning behind checks-effects-interactions becomes clear once you walk through how a reentrancy attack actually works. Suppose a withdrawal function sends the caller's full balance first, and only sets their recorded balance to zero afterward. An attacker's malicious contract includes a fallback function code that automatically executes when the contract receives funds. That fallback function calls the withdrawal function again, before the original call has had a chance to zero out the balance. Because the balance has not yet been updated, the check that would normally prevent a second withdrawal still passes, and funds are sent again. The attacker repeats this cycle within a single transaction, draining the contract's funds again and again. The fix is exactly the checks-effects-interactions ordering: check the caller's balance, then immediately zero it out (update state), and only then send the funds by the time any external call happens, the internal state has already been updated, so a reentrant call finds nothing left to withdraw.

Two further practices reduce risk substantially. First, use audited libraries standardized, battle-tested code such as OpenZeppelin's rather than writing common logic like token standards from scratch, since widely used libraries have already had many independent eyes looking for flaws. Second, use fuzz testing and bug bounties: automated, random-input testing probes edge cases a developer might not think to test manually, while bug bounty programs pay independent researchers to responsibly disclose vulnerabilities they find, rather than exploit them.

How far can audits go? Formal verification is a mathematical approach that proves a contract behaves according to its specification under all possible conditions, offering stronger guarantees than testing alone, but at significantly greater cost and complexity than a standard audit. Even so, no audit is a guarantee: audits significantly reduce risk, but they cannot offer absolute safety, because novel attack vectors and purely economic exploits (as opposed to code-level bugs) can still affect even thoroughly audited contracts. Vulnerabilities located in third-party libraries or in external oracles also frequently fall entirely outside a given audit's scope. The essential lesson is that "audited" is not synonymous with "safe" it describes meaningful risk reduction, not risk elimination.

Privacy-Preserving Techniques

Blockchain's transparency is valuable for auditability, but that same transparency is costly for personal privacy every technique in this section addresses that same underlying tension in a different way.

Zero-knowledge proofs (ZKPs) let one party prove a statement is true without revealing anything else about it. The clearest example: a user can prove they have sufficient funds for a transaction without revealing their actual account balance. zk-SNARKs (Zero-Knowledge Succinct Non-interactive Arguments of Knowledge) are widely used in privacy coins and in Layer 2 rollups. zk-STARKs are a more recent variant, often used to compactly prove the correctness of large batches of off-chain computation a connection worth noting explicitly, since it links directly to the Layer 2 scaling discussion later in this chapter and in Chapter 6.

Beyond zero-knowledge proofs, several other techniques exist. Ring signatures obscure which specific member of a group actually authorized a given transaction, and are used by privacy-focused coins such as Monero. Confidential transactions encrypt the amount involved in a transaction while still allowing the network to verify that no new value was secretly created out of thin air. Mixing services combine multiple users' transactions together, obscuring the link between a specific sender and a specific recipient but these attract significant regulatory scrutiny, because the same tool that protects legitimate privacy can equally be used to launder illicit funds. This dual-use nature is precisely why privacy technology in general draws sustained regulatory attention, even when the underlying cryptography is entirely neutral.

Scalability and Security: A Persistent Tension

Many scaling solutions introduce new trust assumptions, or entirely new categories of vulnerability, alongside the throughput gains they offer. With Layer 2 rollups, a malicious or simply faulty rollup operator could submit an invalid state update – a risk category that does not exist at all on the base layer, where transactions are validated directly by the underlying consensus mechanism. With sharding, splitting data and processing across parallel sub-networks introduces new cross-shard communication challenges, and with them, new security considerations that a single, unsharded chain does not have to contend with.

This tension connects directly back to the blockchain trilemma introduced in Chapter 4: security researchers must continually ask not just “does this scale?” but also “what new way could this be broken?” – improving one property of a system, in this case scalability, routinely opens up new questions about another, in this case security.

Worked Example: Spotting a Reentrancy Vulnerability

1. A withdrawal function sends the caller’s full balance first, then sets their recorded balance to zero afterward – the vulnerable ordering.
2. An attacker’s malicious contract has a fallback function that triggers automatically upon receiving funds.
3. That fallback function calls the withdrawal function again – before the balance was actually zeroed out the first time.
4. The attacker repeats this within a single transaction, draining funds again and again.
5. Fix: apply checks-effects-interactions – check the balance, zero it out, and only then send the funds, in that exact order.

This mirror, in simplified form, the actual mechanism behind the DAO hack described at the start of this chapter.

Chapter 8 Recap: Key Takeaways

- The DAO hack demonstrates a genuine tension between strict immutability and the practical need to respond to serious security incidents – a tradeoff the community resolved through a hard fork, though not without lasting disagreement.

- Reentrancy, integer overflow/underflow, access control flaws, front-running, oracle manipulation, and gas-based denial of service are the most common vulnerability classes affecting smart contracts.
- Audits (both manual and automated), audited libraries, fuzz testing, and bug bounties all meaningfully reduce risk but no audit offers an absolute guarantee of safety.
- Zero-knowledge proofs and related techniques let users prove statements, or transact, without revealing everything to a fully public ledger, addressing blockchain's inherent tension between transparency and privacy.

Discussion Questions and Model Answers

1. Does the DAO hack illustrate a failure of blockchain technology, or a failure of one specific contract's code? Explain your reasoning.

The DAO hack is best understood as a failure of one specific contract's code, not a failure of blockchain technology or of Ethereum's underlying protocol. The vulnerability exploited was a reentrancy flaw in The DAO's own smart contract logic specifically, sending funds before updating internal state rather than any flaw in Ethereum's consensus mechanism, its cryptography, or its network security. Ethereum itself continued operating exactly as designed throughout the incident; what the episode really exposed was a social and governance question how a decentralized community should respond when contract-level bugs cause serious harm rather than a technical failure of the blockchain itself.

2. Describe the mechanics of a reentrancy attack and explain how checks-effects-interactions mitigates the risk.

A reentrancy attack exploits a function that sends funds out before updating its own internal record of the caller's balance. A malicious contract's fallback function, triggered automatically when it receives funds, calls the vulnerable withdrawal function again before the first call has finished updating state, allowing the same balance to be withdrawn repeatedly within a single transaction. Checks-effects-interactions mitigates this by reordering the function: check the caller's eligibility and balance first, update the contract's internal state (zero the balance) second, and only then send the funds. Because the state is already updated by the time any external call is made, a reentrant call finds nothing left to withdraw, closing the vulnerability.

3. How could a zero-knowledge proof verify someone's eligibility for a service without revealing the personal data behind it?

A zero-knowledge proof allows a person to mathematically prove that a statement about their private data is true—for example, “I am over 18” or “my income exceeds the required threshold”—without revealing the actual underlying data, such as their exact birth date or exact income figure, to the party checking eligibility. The verifying party (a service, a smart contract, or another user) can confirm the proof is valid and grant access accordingly, gaining certainty that the eligibility condition genuinely holds, while learning nothing else about the individual beyond the single fact the proof was designed to establish.

Chapter 9: Tokenization and Initial Coin Offerings (ICOs)

Token Standards · ICOs · DAOs · NFTs

Learning Objectives

By the end of this chapter, you should be able to:

- Explain tokenization and the three properties it gives an asset: instant transferability, fractional ownership, and programmable behavior.
- Distinguish the ERC-20, ERC-721, and ERC-1155 token standards, and match each to an appropriate use case.
- Describe how an ICO works, the risks it introduced, and the regulated alternatives (STOs, IEOs) that emerged in response.
- Explain what a DAO is, and identify the practical governance challenges DAOs face.
- Explain what NFTs represent, where they are applied beyond digital art, and what problems they do not solve on their own.

What Is Tokenization? Turning Ownership into Code

Tokenization means representing ownership of, or rights to, an asset—whether cryptocurrency-native, tied to something in the real world such as real estate, or entirely abstract such as voting power—as a digital token on a blockchain. Tokenizing an asset in this way confers three properties that the underlying asset does not have on its own.

Instant transferability allows near-instant movement of ownership, anywhere in the world, without the delays typical of traditional asset transfer processes.

Fractional ownership allows a single asset to be divided and shared among many holders, something that is often impractical for a physical asset like a building but trivial for a token representing a share of one.

Programmable behavior allows a smart contract to automate the rules governing how a token behaves—for instance, automatically enforcing a vesting schedule, or automatically distributing dividends.

Token Standards: ERC-20, ERC-721, ERC-1155

ERC-20 fungible tokens. Each unit is interchangeable with any other unit, exactly like units of a currency: one unit of a given ERC-20 token is worth exactly the same as any other unit of that same token. ERC-20 underlies the vast majority of cryptocurrencies issued on Ethereum.

ERC-721 non-fungible tokens (NFTs). Each token is unique and non-interchangeable, which makes this standard well suited to digital art, collectibles, and unique in-game items, where the whole point is that one unit is not the same as another.

ERC-1155 the multi-token standard. A single contract can manage multiple fungible and non-fungible token types together, which is particularly efficient for something like a video game inventory that needs to represent both interchangeable resources (currency, ammunition) and unique items (a specific weapon skin) within one system.

Initial Coin Offerings (ICOs): How an ICO Works

An ICO typically unfolds in three stages. First, the project publishes a white paper describing its technology, its intended use case, and its “tokenomics” the economic design of the token itself. Second, during the token sale, investors send cryptocurrency to a smart contract in exchange for newly issued tokens, often at a discounted price relative to what the tokens are expected to be worth later. Third, the capital raised funds the project’s development, often well before the underlying product or platform actually exists in a usable form. ICOs became enormously popular in 2017–2018, letting startups raise capital quickly and with far fewer barriers than traditional venture funding required.

The risks. ICOs frequently bypassed established securities regulations altogether. The 2017–2018 boom saw a substantial number of fraudulent projects, weak or nonexistent business models, and significant losses for investors when projects simply failed to deliver on their white papers.

The regulatory response. Regulators the U.S. Securities and Exchange Commission prominent among them argued that many tokens sold through ICOs met the legal definition of a security, and should therefore have been subject to securities law from the start. This prompted the emergence of more regulated alternatives: Security Token Offerings (STOs), explicitly structured to comply with securities regulation from the outset, and Initial Exchange Offerings (**IEOs**), vetted

and hosted by an established cryptocurrency exchange before being offered to investors, adding a layer of due diligence that unregulated ICOs typically lacked.

Decentralized Autonomous Organizations (DAOs): Governance Encoded in Smart Contracts

A DAO is an organization governed by rules encoded in smart contracts, rather than by a traditional management hierarchy. Members hold governance tokens that grant them voting rights proportional to their holdings, and vote outcomes are frequently executed automatically and transparently on-chain, without requiring a board or executive to carry out the decision manually. The underlying ambition is to apply blockchain's core decentralization principles to organizational governance itself, reducing an organization's reliance on traditional corporate hierarchies.

DAOs also face real, unresolved practical challenges. Voter apathy and governance capture are closely related risks: low participation in votes tends to concentrate effective influence among a small number of large token holders, who may end up steering decisions even without a formal majority of all members actually voting.

Legal ambiguity remains a genuine issue, since DAO liability and regulatory classification are unsettled in many jurisdictions – it is often unclear who, legally, is responsible if a DAO's actions cause harm.

Manipulation-resistant design is a genuinely difficult technical and economic problem in its own right: designing a voting mechanism that cannot be gamed by, for example, temporarily acquiring large amounts of a governance token just before a vote, is far from trivial.

Non-Fungible Tokens (NFTs) and Their Applications: Beyond Digital Art

Every application described here builds on the ERC-721 standard introduced above – uniqueness is the common thread running through all of them.

Supply chain. NFTs can represent unique physical items, enabling provenance tracking from manufacture all the way through to final sale.

Gaming. Players can genuinely own their in-game items, which when built on a token standard multiple platforms support become tradeable across those platforms, rather than being locked permanently inside a single game's database.

Identity and credentialing. NFTs can be used to issue verifiable, tamper-resistant academic certificates or professional licenses, whose authenticity anyone can check directly against the blockchain record.

What NFTs don't solve on their own. An NFT typically references its actual content an image file, for example via a metadata link, rather than storing that content directly on-chain, which raises real questions about the content's long-term availability: if the linked file disappears from wherever it was hosted, the NFT itself remains, but the content it was meant to represent may not. NFTs also share blockchain's broader challenges, including significant price volatility and, in some sectors, speculative excess that has become disconnected from any genuine underlying utility.

Worked Example: Designing a Simple Tokenomics Model

1. Total supply. Set a fixed cap for example, 100 million tokens establishing scarcity from the outset.

2. Distribution. A typical split might allocate 40% to investors through the token sale, 20% to the founding team under a multi-year vesting schedule (so the team cannot immediately sell their entire allocation), 20% to a community treasury governed by a DAO, and 20% to broader ecosystem incentives.

3. Utility. What does the token actually do? Common answers include governance voting rights, discounts on platform fees, or being a required form of payment for a specific service the platform provides.

4. Inflationary vs. deflationary design. Are new tokens minted continuously over time (inflationary), or is the supply fixed or even shrinking via a "burn" mechanism that permanently removes tokens from circulation (deflationary)?

5. Risk check. Overly generous allocations to early investors, especially without a meaningful vesting schedule, can create persistent sell pressure on the token's price, as those early holders are free to sell large positions as soon as the token becomes tradeable.

Chapter 9 Recap: Key Takeaways

- Tokenization represents ownership or rights as digital tokens, conferring transferability, programmability, and fractional ownership that the underlying asset does not have on its own.
- ERC-20 (fungible), ERC-721 (non-fungible), and ERC-1155 (multi-token) are the core standards enabling token interoperability across wallets, exchanges, and applications.
- ICOs enabled rapid, low-barrier fundraising but also significant fraud and investor loss, which prompted the emergence of more regulated alternatives such as STOs and IEOs.
- DAOs and NFTs extend tokenization into governance and unique digital ownership respectively both carry genuine promise alongside real, still-unresolved challenges.

Discussion Questions and Model Answers

1. Explain the key difference between the ERC-20 and ERC-721 token standards, with a real-world use case best suited to each.

ERC-20 tokens are fungible: every unit is interchangeable with every other unit, exactly like units of a currency, which makes ERC-20 the standard best suited to a use case such as a platform's own utility token or a stablecoin, where every token needs to be worth precisely the same as any other. ERC-721 tokens are non-fungible: each individual token is unique and cannot be exchanged one-for-one with another, which makes ERC-721 the standard best suited to a use case such as digital art or collectibles, where the entire value proposition depends on each item being distinct and individually identifiable rather than interchangeable.

2. Describe the typical mechanics of an ICO and explain why this fundraising model attracted significant regulatory scrutiny.

A typical ICO begins with a project publishing a white paper describing its technology, use case, and tokenomics; investors then send cryptocurrency to a smart contract during a token sale in exchange for newly issued tokens, often at a discount; the funds raised finance the project's

development, frequently well before any working product exists. This model attracted significant regulatory scrutiny because it frequently bypassed established securities regulations, and because the 2017–2018 boom included a substantial number of fraudulent projects and weak business models that caused significant losses for ordinary investors prompting regulators such as the U.S. SEC to argue that many of these tokens met the legal definition of a security and should have been regulated accordingly from the start.

3. What governance challenges does the DAO organizational model face in practice, and why do they matter?

DAOs face voter apathy and governance capture, where low participation in votes concentrates effective influence among a small number of large token holders, undermining the model's aspiration toward broad-based, decentralized decision-making. They also face legal ambiguity, since DAO liability and regulatory classification remain unsettled in many jurisdictions, creating real uncertainty about who is legally responsible when a DAO's decisions or actions cause harm. Finally, they face the genuinely difficult technical and economic challenge of designing voting mechanisms that resist manipulation, such as vote-buying or the temporary acquisition of governance tokens purely to sway a single vote. These challenges matter because they go directly to whether DAOs can deliver on their core promise governance that is meaningfully more decentralized and accountable than a traditional corporate hierarchy or whether, in practice, they simply relocate familiar governance problems onto a blockchain rather than solving them.

Chapter 10: Use Cases of Blockchain in Various Industries

Finance · Healthcare · Supply Chain · Governance

Learning Objectives

By the end of this chapter, you should be able to:

- Explain how decentralized exchanges and lending protocols reimagine core financial services, and how blockchain reshapes cross-border payments and trade finance.
- Describe blockchain's proposed role in healthcare, from unified patient records to pharmaceutical supply chains and clinical trials.
- Explain the IBM Food Trust case study and generalize its pattern to other supply chain applications.
- Describe blockchain's applications in governance, including land registries, self-sovereign identity, and the specific tensions that keep blockchain voting largely experimental.
- Apply a cross-industry framework to judge when blockchain genuinely helps a given problem, and when it does not.

The chapters up to this point built the technical and economic foundations of blockchain; this chapter is where all of that machinery becomes concrete. It surveys real industries, real deployments, and real lessons about where blockchain genuinely helps and, just as importantly, where it does not.

8.1 Finance and Decentralized Finance (DeFi)

Decentralized exchanges (DEXs). Platforms such as Uniswap let users trade tokens directly from their own wallets using automated market makers (AMMs) trades execute against a shared liquidity pool of two paired assets, rather than being matched against a traditional order book of individual buy and sell orders. DeFi represents the most extensive real-world deployment of the smart contract concepts introduced in earlier chapters: instead of a human market maker or a centralized exchange matching a buyer with a seller, a smart contract automatically prices and executes trades against the pool according to a fixed mathematical formula.

Lending protocols. Protocols such as Aave and Compound let users deposit cryptocurrency to earn interest, or borrow against crypto they have already deposited as collateral. Interest rates are set algorithmically by the protocol itself, with no credit check and no bank involved anywhere in the process.

Cross-border payments. Blockchain-based settlement can significantly reduce both the cost and the time of international transfers, compared to conventional correspondent banking networks, which typically route a payment through several intermediary banks, each adding delay and fees.

Trade finance. Smart contracts can automate the verification and release of payments tied to the fulfillment of contractual shipping and delivery conditions—for instance, automatically releasing payment to a supplier once a shipment is verifiably confirmed as delivered, removing the need for manual document-checking at every stage. This category broadens the picture beyond consumer-facing DeFi applications: these are institutional and business-to-business applications, illustrating that blockchain's reach extends into how banks and traders actually operate, not only into retail cryptocurrency use.

8.2 Healthcare

The problem: fragmented records. Patient health records are traditionally scattered across multiple providers and institutions, each maintaining its own separate, often incompatible system making it difficult for a patient's complete medical history to travel with them from one provider to another.

The blockchain proposal. Give patients a unified, cryptographically secured record of their own medical history, with the patient themselves—rather than any single institution—controlling which providers are granted access to which parts of that record. This addresses interoperability (getting different systems to work together) and patient data sovereignty (giving patients real control over their own data) simultaneously. Patient data sovereignty here is really the healthcare-specific application of a broader idea that reappears later in this chapter under governance: self-sovereign identity.

Pharmaceutical supply chain. Tracking medications from manufacture through distribution to final dispensing helps combat counterfeit drugs, by creating an immutable, auditable record of

provenance at every stage of the supply chain. Consider how one would verify a medication's authenticity today, without such a system, versus with a blockchain-based provenance record attached to every step the drug has passed through the counterfeit drug problem is a powerful, concrete motivator for this kind of application.

Clinical trials and research. Blockchain can securely manage informed consent in clinical trials, and can enable auditable sharing of anonymized health data for research purposes while still preserving patient privacy a point that connects directly back to the zero-knowledge proof discussion in Chapter 8, since ZKPs are well suited to exactly this kind of privacy-preserving data sharing.

8.3 Supply Chain Management

Case study: IBM Food Trust. Built on Hyperledger Fabric, IBM Food Trust connects retailers and suppliers on a shared, tamper-resistant ledger that tracks food products from farm to shelf. The before-and-after contrast is the single most memorable statistic in this chapter.

Before: tracing a product's origin during a contamination outbreak could take days to weeks under paper-based or siloed digital record-keeping, as investigators worked backward through disconnected systems maintained by different companies.

After: retailers and suppliers can trace a product back to its source farm within seconds, dramatically accelerating food safety responses and potentially saving lives during an active outbreak.

Beyond food. The same underlying pattern generalizes well beyond food safety.

Anti-counterfeiting applies it to combating counterfeiting in luxury goods, by verifying a product's authentic chain of custody from manufacturer to buyer.

Ethical sourcing applies it to verifying claims such as conflict-free diamonds or sustainably sourced cobalt and cocoa.

Agricultural provenance applies it to improving transparency in the supply chains of commodities such as coffee and cocoa more broadly, beyond food-safety concerns specifically. The common thread across all of these: many independent parties need to share and verify a single,

consistent, trustworthy record of where something came from – precisely the kind of multi-party trust gap blockchain is well suited to close, especially when a genuine trust gap exists between participants and the supply chain crosses multiple companies or borders.

8.4 Governance and Public Administration

Land registries. Tamper-resistant, transparent property ownership records aim to reduce fraud and disputes over who actually owns a given piece of property. This approach has been piloted in Georgia, Sweden, and parts of India, with varying degrees of success and adoption – a useful reminder that a technically sound idea does not automatically translate into a successful large-scale deployment.

Self-sovereign identity. Individuals hold verifiable credentials – proof of age, or proof of a qualification, for example – in a personal digital wallet, and can selectively share specific verified attributes without revealing unnecessary additional personal information in the process. This connects directly to the zero-knowledge proof discussion in Chapter 8: selective disclosure of individual attributes, without revealing everything else about a person, is exactly the kind of problem ZKPs are well suited to solve.

Why blockchain voting remains experimental. Blockchain voting could, in principle, improve the transparency and auditability of elections – but it must satisfy several difficult requirements simultaneously, and some of those requirements are genuinely in tension with one another.

Voter privacy requires that no one should be able to see how any individual voted.

Coercion resistance requires that a voter should not be able to prove to someone else how they voted, even if they wanted to – otherwise a voter could be pressured or paid to vote a certain way and then forced to prove compliance.

Public verifiability requires that anyone should be able to verify the final count is correct. Achieving privacy and public verifiability at the same time is already a hard-cryptographic problem; adding coercion resistance on top makes it harder still, since a system that lets a voter privately prove their own vote to themselves (for verification purposes) can often be adapted by a coercer to force that same proof to be shown to them instead. Combined with ongoing concerns about the security of voters’ personal devices during an actual election, this is why blockchain

voting remains predominantly experimental rather than deployed for high-stakes national elections.

8.5 Cross-Industry Lessons: When Does Blockchain Actually Help?

This section is the conceptual capstone of the chapter, and arguably of the module's applied portion as a whole.

Blockchain fits well when multiple independent parties do not fully trust one another, but nonetheless need to share and verify a common set of facts the five competing shipping companies from Chapter 5's worked example are a clean illustration of exactly this condition.

Blockchain fits poorly when a single trusted authority already exists and functions effectively, since in that case the overhead of decentralization may simply outweigh any benefit it could provide replacing a well-functioning centralized system with a decentralized one purely for its own sake tends to add complexity without adding real value.

Successful real-world deployments also tend to share a second characteristic worth emphasizing: they integrate carefully with existing off-chain systems, rather than attempting to replace an entire industry's infrastructure wholesale and all at once. IBM Food Trust did not replace grocery retailers' or suppliers' existing operations; it added a shared, trusted ledger layer on top of them. This pragmatic, incremental pattern solves a specific multi-party trust problem, integrate carefully with what already exists is a useful lens for evaluating any new proposed blockchain application, including in student projects and assignments.

Chapter 10 Recap: Key Takeaways

- DeFi recreates core financial services exchanges, lending without traditional intermediaries, while blockchain also reshapes institutional processes like cross-border payments and trade finance.
- Healthcare blockchain applications target fragmented patient records, counterfeit pharmaceuticals, and privacy-preserving clinical research.
- Supply chain management is one of blockchain's most commercially mature use cases IBM Food Trust cut traceability time from days or weeks down to seconds.

- Blockchain delivers the most genuine value precisely when multiple distrusting parties must share verifiable facts together not simply because a given problem happens to involve data that could technically be put on a blockchain.

Discussion Questions and Model Answers

1. How does an automated market maker differ from a traditional order-book exchange when facilitating a trade?

A traditional order-book exchange matches individual buy and sell orders from different market participants directly against one another, requiring sufficient counterparties on both sides at compatible prices for a trade to execute. An automated market maker instead executes trades directly against a shared liquidity pool holding a pair of assets, with prices determined algorithmically by a fixed mathematical formula based on the pool's current balance of each asset, rather than by matching individual human orders. This means a trade can execute immediately against the pool even without another individual trader on the opposite side at that exact moment, at a price the formula determines automatically.

2. Using the IBM Food Trust example, explain why rapid traceability is particularly valuable during a food safety incident.

During a contamination outbreak, every day spent tracing a product's origin through paper-based or siloed digital records is a day during which contaminated products may remain on shelves, in warehouses, or in consumers' homes, actively posing a health risk. IBM Food Trust's shared, tamper-resistant ledger allows retailers and suppliers to trace a product back to its specific source farm within seconds instead of days or weeks, allowing recalls and public health responses to happen far faster directly reducing the window during which contaminated products can cause further harm.

3. Identify a business problem not discussed in this chapter, and assess whether blockchain would offer a genuine advantage over existing systems.

Consider academic transcript verification for international university applications. Multiple independent parties the issuing university, the receiving university, and often a third-party credential verification service currently do not fully trust one another's paper or PDF transcripts

without an additional (often slow and costly) manual verification step. Applying the cross-industry framework from this chapter: this looks like a reasonable candidate for blockchain, since it involves multiple parties that do not fully trust each other needing to verify a shared fact (that a transcript is genuine and unaltered), similar in spirit to the hybrid-blockchain university transcript example discussed in Chapter 5, where only a cryptographic hash of the transcript needs to be published for verification, while the transcript's actual private content stays confidential. The caveat, per the same framework, is that this only offers a genuine advantage if issuing universities are willing to adopt the system in the first place – a single university's own existing verification process may already function well enough that the overhead of an entirely new blockchain-based system outweighs the benefit, unless enough institutions coordinate to adopt it together.

Appendix A Glossary of Key Terms

This glossary consolidates the key vocabulary introduced across all ten chapters, in the order the terms were first introduced, for quick reference and revision.

Blockchain. A shared, continuously growing record of transactions, copied across many independent computers, in which each new block is cryptographically linked to the block before it.

Block. A bundle of transactions plus metadata: a timestamp, the previous block's hash, and a Merkle root summarizing the block's own transactions.

Chain. The sequence of blocks, each referencing the hash of the block before it, forming a tamper-evident history.

Node. A computer participating in a blockchain network by storing and/or validating the ledger.

Ledger. The complete, continuously updated record of every transaction that has occurred on a network – a structured record of transactions, not simply a storage medium.

Decentralization. Distributing control away from a single authority and across a distributed network of independent participants.

Consensus mechanism. The process by which a network agrees on which transactions and blocks are valid, without a central authority.

Wallet. Software or hardware that stores the cryptographic keys used to access and transact digital assets.

Digital scarcity. The principle that a digital asset has a strictly limited supply, which is part of what gives it value.

Distributed control, transparency, immutability, trustlessness. The four pillars of decentralized systems introduced in Chapter 1.

Centralized / distributed / decentralized. Three distinct concepts: centralized describes who controls a system; distributed describes physical or logical layout; decentralized means authority is genuinely shared across independent participants.

Hash function. A mathematical function converting any input into a fixed-size output (a digest); the security backbone of blockchain.

Avalanche effect. The property by which changing even one character of a hash function's input completely changes its output.

Pre-image resistance. The property by which it is computationally infeasible to reverse a hash output back into its original input.

Merkle tree. A hierarchical structure that combines and re-hashes pairs of transaction hashes until a single Merkle root remains, enabling efficient verification of individual transactions.

Private key. A secret, randomly generated number that proves ownership and authorizes spending; must never be shared.

Public key. A key mathematically derived from a private key, shareable openly, and hashed to form a blockchain address.

Digital signature. Cryptographic proof that a transaction was created by the legitimate owner of a private key, providing authentication, integrity, and non-repudiation.

Mempool. A temporary storage area, held in each full node's memory, containing valid but unconfirmed transactions waiting to be included in a block.

Nonce. A number used once a value in a block header that miners repeatedly change while searching for a hash that meets the difficulty target.

Difficulty target. A large number that a candidate block's hash must fall below, numerically, in order for the block to be accepted; periodically adjusted to keep block time roughly constant.

Byzantine Generals Problem. The classic computer science problem of how multiple participants can reach reliable agreement when some may be unreliable or malicious.

Proof of Work (PoW). A consensus mechanism in which miners compete to find a nonce whose resulting hash meets a difficulty target; used by Bitcoin.

Block reward. Newly created cryptocurrency paid to the miner who successfully adds a valid block to the blockchain.

Halving. A scheduled event, roughly every four years on Bitcoin, that cuts the block reward in half, capping total supply and shifting miner revenue toward transaction fees over time.

51% attack. An attack in which a single party or coalition controls more than half of a network's mining power, potentially allowing them to reverse their own recent transactions.

Proof of Stake (PoS). A consensus mechanism in which validators lock up ("stake") cryptocurrency as collateral rather than performing computational work; used by Ethereum since 2022.

Slashing. The penalty in Proof of Stake in which a dishonest or rule-violating validator forfeits part or all of their staked funds.

Delegated Proof of Stake (DPoS). A consensus model in which token holders elect a small number of delegates to validate transactions on their behalf.

Practical Byzantine Fault Tolerance (PBFT). A consensus mechanism, common in permissioned networks, in which a known set of validators exchange messages to reach immediate, final agreement.

Proof of Authority (PoA). A consensus mechanism in which a fixed set of pre-approved, identity-verified validators produce blocks.

Blockchain trilemma. The observation that decentralization, security, and scalability are difficult for any blockchain to fully maximize simultaneously.

Public blockchain. A fully open, permissionless network that anyone can join, read, and validate; e.g., Bitcoin, Ethereum.

Private blockchain. A blockchain controlled by a single organization, which decides who may join and how validation works.

Consortium (federated) blockchain. A blockchain governed collectively by a pre-selected group of organizations.

Hybrid blockchain. A blockchain combining private and public elements sensitive data stored privately, with a cryptographic hash published publicly for verification.

Ethereum Virtual Machine (EVM). The Turing-complete runtime that executes smart contracts on every Ethereum node.

Gas. A measure of the computational work required by an EVM operation, paid for in ETH.

Layer 2 / rollup. A secondary network that handles most transaction execution off the main chain, then settles results back to Layer 1, inheriting its security guarantees.

Smart contract. Self-executing code, deployed on-chain, that automatically enforces an agreement once its conditions are met the “vending machine” concept.

Solidity. The most widely used high-level programming language for writing Ethereum smart contracts.

ABI (Application Binary Interface). A JSON description of a smart contract’s functions, enabling a frontend to construct valid calls to the contract.

dApp (decentralized application). An application typically composed of a smart contract layer, a connecting layer (e.g., Web3.js), and a frontend layer.

Reentrancy attack. An attack in which a malicious contract repeatedly calls back into a vulnerable function before its state updates complete, draining funds through repeated withdrawals.

Checks-effects-interactions. The standard defensive coding pattern: perform checks first, update internal state second, and make external calls last the primary defense against reentrancy.

Zero-knowledge proof (ZKP). A cryptographic method allowing one party to prove a statement is true without revealing any other information.

Tokenization. Representing ownership of, or rights to, an asset as a digital token on a blockchain.

ERC-20. The Ethereum token standard for fungible (interchangeable) tokens.

ERC-721. The Ethereum token standard for non-fungible tokens (NFTs), each of which is unique.

ERC-1155. The Ethereum multi-token standard, allowing a single contract to manage multiple fungible and non-fungible token types.

Initial Coin Offering (ICO). A fundraising method in which a project sells newly created tokens directly to investors, typically before its product is fully built.

DAO (Decentralized Autonomous Organization). An organization governed by rules encoded in smart contracts, with members voting via governance tokens.

Decentralized Finance (DeFi). Blockchain-based financial services trading, lending, borrowing operating without traditional intermediaries such as banks.

Automated Market Maker (AMM). A mechanism, used by decentralized exchanges, that executes trades against a shared liquidity pool rather than matching individual buy and sell orders.

Appendix B Quick-Reference Comparison Tables

This appendix collects the module's core comparison tables in one place for revision purposes.

Consensus mechanisms at a glance

	Proof of Work	Proof of Stake	Delegated / Authority
Security Basis	Computational cost	Staked capital + slashing	Known, trusted validators
Energy Use	Very high	Very low	Very low
Speed / Finality	Slower, probabilistic	Faster	Fastest

Decentralization	High	High (stake-dependent)	Lower
------------------	------	------------------------	-------

Blockchain network types at a glance

	Public	Private	Consortium	Hybrid
Who Can Join	Anyone	One organization	Pre-selected group	Mixed access
Control	Fully decentralized	Fully centralized	Shared among members	Partially shared
Transaction Speed	Slower	Fastest	Fast	Variable
Transparency	Fully public	Fully private	Private to members	Selective

Major platforms at a glance

	Bitcoin	Ethereum	Hyperledger Fabric
Network Type	Public, permissionless	Public, permissionless	Permissioned
Smart Contracts	Limited scripting	Full (Turing-complete)	Full (“chaincode”)
Primary Use Case	Digital cash / store of value	DeFi, NFTs, general dApps	Enterprise / consortium
Consensus	Proof of Work	Proof of Stake	Configurable

Token standards at a glance

Standard	Type	Typical Use Case
ERC-20	Fungible	Cryptocurrencies, utility tokens, stablecoins
ERC-721	Non-fungible	Digital art, collectibles, unique in-game items
ERC-1155	Multi-token (fungible + non-fungible)	Gaming inventories combining currency and unique items

Appendix C Additional Self-Check Questions

Use these questions, organized by chapter, to test your understanding beyond the in-chapter discussion questions. They are intended for self-study and seminar discussion; model answers are not provided, so that they can also be used for graded coursework.

Chapter 1. Explain, in your own words, why “trustless” does not mean “no trust is involved.” Give an example of a system that is distributed but not decentralized, other than the ones used in this handout.

Chapter 2. A friend argues that losing a private key is “basically the same” as forgetting a bank password. Explain what is wrong with this comparison. Why does collision resistance matter for Merkle trees specifically?

Chapter 3. Suppose Block 250 is altered five blocks after it was mined. Explain, step by step, what an attacker would need to do to make the altered chain accepted by the rest of the network.

Chapter 4. Explain why Proof of Stake does not need a “difficulty target” the way Proof of Work does. Under what circumstances might a small blockchain reasonably choose Proof of Authority over Proof of Stake?

Chapter 5. A university consortium of five institutions wants to share verified diploma records with employers, without giving any single university control over the shared system. Which network type fits best, and why? What would change about your answer if only one university were involved?

Chapter 6. Explain why Ethereum needed gas, but a permissioned Hyperledger Fabric network typically does not use gas in the same way. What advantage does the Lightning Network offer that upgrading Bitcoin’s base-layer block size alone would not?

Chapter 7. A student wants to build a simple decentralized application that lets users tip content creators in a custom token. Sketch which of the three dApp layers this would require, and identify one design decision that should be made “off-chain” rather than “on-chain.”

Chapter 8. Explain, using the checks-effects-interactions pattern, why a contract that only ever sends funds to `msg.sender` and never to an address supplied by the caller is naturally more resistant to certain attacks. Why is a security audit not the same thing as a formal verification?

Chapter 9. Compare an ICO and an IEO in terms of the level of due diligence a typical investor can expect. Explain why a DAO's governance token distribution might itself create a centralization risk, even though the DAO's decisions are made "on-chain."

Chapter 10. Using the cross-industry framework from Chapter 10, evaluate whether blockchain would offer a genuine advantage for university course-credit transfer between institutions. Identify which of the requirements for blockchain voting (privacy, coercion resistance, public verifiability) is, in your view, hardest to satisfy simultaneously with the other two, and explain your reasoning.

End of MSE 9214 Course Handout.